

MMMC File

Thursday, September 17, 2026 9:58 PM

What is an .MMMC File ?

Multi-Mode Multi-Corner – One File to Rule All Timing Scenarios

Same Design. Different Conditions. Ensure It Always Works!

MMMC connects the dots between libraries, constraints and analysis scenarios!

1 Full Form & Definition

MMMC = Multi-Mode Multi-Corner

- A Tcl script (.mmmc file) that defines all the modes and corners that need to be analyzed in STA and P&R tools.
- It tells the tool which .lib, .sdc and RC (parasitics) files to use for each analysis view.
- It standardizes the timing scenarios and ensures consistent analysis across the design flow.

2 Why It is a Tcl Configuration File ?

- MMMC is written in Tcl because EDA tools (Genus, Innovus, Tempus) use Tcl for configuration.
- Tcl provides a flexible and powerful way to define libraries, corners, modes and analysis views.
- It is a text file with commands (e.g., create_library_set, create_constraint_mode, create_analysis_view).
- Tool reads this file and automatically sets up all the required timing analysis runs.

Example .mmmc snippet

```
# Create library set for SS corner
create_library_set -name LS_SS \
  -timing { ./pdk/lib/ss_0p72v_125c.lib }
# Create constraint mode for functional mode
create_constraint_mode -name FUNC \
  -sdc_files { ./sdc/func.sdc }
```

3 Purpose of MMMC

- To verify that the design works correctly under all process, voltage, temperature (PVT) variations.
- To handle multiple operating modes (functional, test, low power, boot, etc.).
- To automate and manage a large number of timing analysis runs.
- To provide a single, consistent setup for STA and P&R.
- To improve accuracy, reliability and signoff confidence.

4 What Problem Does MMMC Solve ?

- Every chip comes out slightly different due to process variation.
- Voltage and temperature also vary in real silicon.
- Chip operates in different modes with different clocks and constraints.
- Manually running each combination is error-prone and inconsistent.

MMMC solves this by defining all mode + corner combinations in one file, so the tool can automatically run and analyze them.

Without MMMC

- Manual setup for each run
- High chance of human error
- Different engineers may use different setups
- Inconsistent results

With MMMC

- One file for all scenarios
- Automatic and consistent runs
- Correct libraries, SDC and RC for each view
- Saves time and improves timing signoff quality

5 Relationship with STA and P&R

.MMMC File (Defines modes, corners, libraries, SDC, RC)

STA (Tempus)

- Reads .mmmc
- Performs multi-mode multi-corner timing analysis
- Reports timing across all analysis views

P&R (Innovus)

- Reads .mmmc
- Uses same views for timing-driven placement, CTS and routing
- Ensures consistency with STA

6 Overall Flow in Design

.lib (PVT) → .sdc (Mode) → .RC (QRC/SPEF) → .mmmc (Combines All)

↓

STA / P&R Tools (Genus / Innovus / Tempus)

↓

Signoff (Design works in all modes and corners)

Key Takeaway: MMMC is the **timing analysis blueprint** of the design. It defines all operating modes and PVT corners, and ensures that both STA and P&R tools use the correct libraries, constraints and parasitics for each scenario.

"Right Analysis Today. Reliable Silicon Tomorrow!"

Why Do We Need MMMC in a Real Chip ?

Because one design must work correctly in all modes and all PVT conditions !

Different Chips Different Conditions Same Design!

1 Silicon Variation

- Due to manufacturing variations, every chip comes out slightly different.
- Transistors can be faster or slower than nominal depending on process variation.
- So the same design and layout can have different timing in different chips.

Same Design Same Layout

Fast chip (FF) ~20% faster than TT

Typical chip (TT) Nominal speed

Slow chip (SS) ~20% slower than TT

2 Multiple PVT Conditions

Timing of a chip depends on Process, Voltage and Temperature (PVT).

PROCESS	VOLTAGE	TEMPERATURE
FF Faster than nominal (Vth lower)	0.88 V Higher voltage → faster switching → more power	-40°C Cold (wire R lower, cells can be faster)
TT Nominal (what PDK targets)	0.80 V Nominal voltage	25°C Room (nominal)
SS Slower than nominal (Vth higher)	0.72 V Lower voltage → slower switching → less power	125°C Hot (wire R higher, cells slower)

All three (Process, Voltage and Temperature) vary simultaneously in silicon, so the design must work correctly across all combinations.

3 Multiple Operating Modes

- A chip works in different modes, each with different clock frequencies, active blocks and constraints.
- Each mode needs a separate SDC and may have different timing paths.

Mode	Clock Frequency	Key Characteristics
Functional	1 GHz	Normal operation All clocks active
Low Power	500 MHz	Reduced voltage Some domains OFF
Test (Scan)	100 MHz	Scan shift mode (scan_en = 1)
Boot	200 MHz	Only boot ROM and PMU Most clocks gated

4 Why Multiple Modes × Multiple Corners ?

STA must verify the design for every mode at relevant PVT corners (setup, hold and sometimes typical).

N modes × M corners = Total analysis runs

4 Modes × 6 corners (example) = 24

4 Modes

- Functional
- Low Power
- Test (Scan)
- Boot

6 Corners (example)

- SS @ 0.72V, 125°C (setup worst)
- FF @ 0.88V, -40°C (hold worst)
- TT @ 0.80V, 25°C (nominal)
- SF (optional)
- FS (optional)
- Extra RC/derate corner (optional)

Example Analysis Combinations

- FUNC @ SS (setup)
- FUNC @ FF (hold)
- LP @ SS (setup)
- LP @ FF (hold)
- TEST @ TT
- BOOT @ SS
- ... and more

5 Problem with Manual Setup

- Engineer has to manually configure each run with correct .lib, .sdc and RC.
- High chance of human error (wrong library, wrong SDC, wrong corner).
- Different engineers may use different setups → inconsistent results.
- Very time consuming for large number of runs.

Without a central MMMC file: Manual runs are error-prone, inconsistent and hard to manage.

6 How MMMC Solves It

- Defines all modes, corners, libraries, SDC and RC in a single Tcl file.
- Tool automatically creates all required analysis views.
- Ensures consistent setup across STA and P&R tools (Tempus, Innovus, etc.).
- Improves accuracy, reliability and saves a lot of time.

MMMC = One file. All scenarios. Consistent results. Project ready!

Key Takeaway: We need MMMC because a real chip must work correctly in multiple operating modes and across all process, voltage and temperature variations, and manually running every combination is not practical.

"Coverage Today Avoids Silicon Surprises Tomorrow!"

What is a PVT Corner ?

A combination of Process, Voltage and Temperature conditions that defines how fast or slow a chip can operate

Same Design Different PVT Different Speed !

1 Definition

PVT Corner = Process + Voltage + Temperature

- A specific combination of process, supply voltage and temperature conditions.
- It represents a real operating condition in which the chip must function correctly.
- Each corner has different cell delays, power and leakage characteristics.
- EDA tools (Genus, Innovus, Tempus) use PVT

2 The Three Components of PVT

Process (P)	Voltage (V)	Temperature (T)
- Variation in manufacturing process. - Affects transistor parameters (Vth, mobility, oxide thickness, etc.).	- Supply voltage can vary due to IR drop, regulator tolerance, and operating modes. - Higher voltage → faster execution	- Operating temperature varies with environment and workload. - Higher temperature increases resistance and slows down

3 Common Values

Process Corners

Corner	Meaning
FF	Fast-Fast (faster than nominal)
TT	Typical-Typical (nominal)
SS	Slow-Slow (slower than nominal)
SF	Slow NMOS, Fast PMOS (optional)
FS	Fast NMOS, Slow PMOS (optional)

Voltage Corners

Voltage	Meaning
0.72 V	Low voltage (slower, less power)

corners to analyze and signoff the design.

- Different chips on the same wafer can be faster or slower.
- Lower voltage → slower switching, less power.
- Lower temperature can make transistors faster (depending on technology node).

4 Why Each Varies Independently ?

- Process**: Set by manufacturing (different wafers/chips) → Cannot be controlled by the designer
- Voltage**: Varies due to power supply, IR drop, regulator tolerance and operating modes → Varies during real operation
- Temperature**: Depends on environment, workload, and packaging → Changes across use cases

All three (P, V and T) can change at the same time and are independent of each other.

5 How PVT Affects Cell Delay and Power

Factor	Effect on Cell Delay	Effect on Power
Process (FF vs SS)	FF → faster (lower delay) SS → slower (higher delay)	FF → higher dynamic power SS → lower dynamic power (less leakage at same V)
Voltage (high vs low)	Higher V → lower delay Lower V → higher delay	Higher V → higher power Lower V → lower power
Temperature (cold vs hot)	Lower T → lower delay (usually faster) Higher T → higher delay (slower)	Higher T → higher leakage Lower T → lower leakage

6 Example PVT Corners and Their Behavior

Corner	Process	Voltage	Temp	Relative Speed	Typical Use
FF	Fast	0.88 V	-40°C	Fastest	Hold check (min delay)
TT	Typical	0.80 V	25°C	Nominal	Functional (typical)
SS	Slow	0.72 V	125°C	Slowest	Setup check (max delay)

Key Takeaways:

- A PVT corner is a specific combination of **Process, Voltage and Temperature**.
- Each factor varies **independently** and affects timing and power differently.
- Design must be verified across all relevant PVT corners to ensure correct operation in silicon.

"Multiple PVT Corners Stronger Designs Real Silicon Confidence!"

PVT Corners

What are SS, TT and FF Process Corners ?

Same design, different silicon behavior due to manufacturing variations

Same Design Same Layout But Different Speed in Silicon!

1 Overview

- Process corner models the effect of manufacturing variations on transistor parameters.
- Transistors are never exactly the same in silicon – their Vth, mobility, oxide thickness, etc. vary.
- Foundry provides different .lib files for different process corners.
- Common process corners: FF (Fast-Fast), TT (Typical-Typical), SS (Slow-Slow).

2 FF Corner (Fast-Fast)

Fastest transistors

Transistors switch faster than nominal.

FF = Fast-Fast
(both NMOS and PMOS are fast)

3 TT Corner (Typical-Typical)

Nominal / Target condition

Represents the typical / expected silicon.

TT = Typical-Typical
(both NMOS and PMOS are typical)

4 SS Corner (Slow-Slow)

Slowest transistors

Transistors switch slower than nominal.

SS = Slow-Slow
(both NMOS and PMOS are slow)

5 Comparison of Transistor Behavior

Parameter	FF (Fast)	TT (Typical)	SS (Slow)
Threshold Voltage (Vth)	Lower	Nominal	Higher
Carrier Mobility	Higher	Nominal	Lower
Oxide Thickness	Thinner	Nominal	Thicker
Drive Strength (Current)	Higher	Nominal	Lower
Cell Delay	Lower (Faster)	Nominal	Higher (Slower)
Leakage Power	Higher	Nominal	Lower
Dynamic Power	Higher	Nominal	Lower

6 Effect on Timing and Power

Timing (Cell Delay)

- FF → minimum delay (fastest)
- TT → nominal delay
- SS → maximum delay (slowest)

Power

- FF → higher dynamic and leakage power
- TT → nominal power
- SS → lower dynamic power (but slower)

Fast transistors give higher performance but consume more power. Slow transistors save power but are slower.

7 Typical Use of Each Corner

Corner	Typical Use in STA / Signoff
FF	• Worst case for Hold analysis (fastest paths, minimum delay) • Also used for power analysis (higher leakage)
TT	• Functional / nominal analysis • Design debugging • Sometimes used for early P&R and power estimation
SS	• Worst case for Setup analysis (slowest paths, maximum delay) • Used for signoff timing (ensure design works in slow silicon) • Lower power corner (less leakage)

Key Takeaway:

- SS, TT and FF are process corners provided by the foundry to model manufacturing variations.
- They represent slow, typical and fast transistor behavior, which affects Vth, drive strength, timing and power.
- STA uses SS for setup, FF for hold, and TT for nominal analysis.

"Different Silicon, Different Speeds, Same Design!"

PVT

How Do Voltage and Temperature Affect Timing ?

Same design, different V and T → different speed in silicon

Lower V Slower Chip
Higher T Slower Chip!

1 Typical Voltage Corners

Supply voltage directly affects transistor drive current.

Voltage	Condition	Effect on Timing	Effect on Power
0.72 V	Low V (-10%)	Slower (higher delay)	Lower power (saves battery)
0.80 V	Nominal	Nominal	Nominal
0.88 V	High V (+10%)	Faster (lower delay)	Higher power (more current)

2 Typical Temperature Corners

Temperature affects both transistors and interconnects.

Temperature	Condition	Effect on Timing	Effect on Wire R
-40°C	Cold	Faster (lower delay)	Lower wire resistance
25°C	Room (Nominal)	Nominal	Nominal
125°C	Hot	Slower (higher delay)	Higher wire resistance

3 Why Voltage Changes Timing ?

- Transistor drive current (Id) increases with higher Vdd.
- Higher voltage → stronger drive → faster switching (lower delay).
- Lower voltage → weaker drive → slower switching (higher delay).
- Delay is roughly inversely related to (Vdd - Vth).

$$\text{Cell Delay} \propto \frac{1}{(V_{dd} - V_{th})^\alpha} \quad (\alpha \approx 1 \sim 2)$$

So, when Vdd decreases, (Vdd - Vth) becomes smaller and cell delay increases significantly.

4 Why Temperature Changes Timing ?

- Higher temperature → lower carrier mobility → slower transistors → higher cell delay.
- Wire resistance increases with temperature → higher interconnect delay.
- At very low temperature (-40°C), transistors are generally faster and wire resistance is lower.
- At high temperature (125°C), both cell and wire delays increase, making the chip slower.

SS corner at 125°C is usually the slowest condition for timing analysis (worst for setup).

5 Effect on Cell Delay (Transistor)

As voltage increases, cell delay decreases (non-linear).

6 Effect on Wire Resistance

Wire resistance (R) increases with temperature.

$$R(T) = R_0 [1 + \alpha (T - T_0)]$$

α (for Cu) = 0.0039 / °C

Temperature	Relative Wire Resistance	Effect on Interconnect Delay
-40°C	~ 0.84 x	Lower delay
25°C	1.0 x	Nominal
125°C	~ 1.39 x	Higher delay

7 Combined Effect on Timing (Example)

Condition	Voltage	Temperature	Cell Delay	Wire Delay	Overall Effect
Best case (fast)	0.88 V	-40°C	Low	Low	Fastest (used for hold)
Nominal	0.80 V	25°C	Nominal	Nominal	Typical
Worst case (slow)	0.72 V	125°C	High	High	Slowest (used for setup)

Key Takeaways

- Higher voltage → faster switching → lower cell delay.
- Lower voltage → slower switching → higher cell delay.
- Higher temperature → slower transistors and higher wire resistance.
- Voltage and temperature both affect timing, but in different ways.
- Worst setup corner: 0.72 V + 125°C (slowest).
- Worst hold corner: 0.88 V + -40°C (fastest).

Right V, Right T, Right Timing for a Reliable Design!

PVT Corners

What are SF and FS Corners, and When are they Required ?

To capture cross-corner effects where NMOS and PMOS do not vary in the same direction

Real Silicon is more complex than just SS / TT / FF!

1 What are SF and FS Corners ?

SF Corner	FS Corner
Slow NMOS / Fast PMOS	Fast NMOS / Slow PMOS
- NMOS transistors are slower (higher V_{th}, lower mobility). - PMOS transistors are faster (lower V_{th}, higher mobility). - Causes unbalanced rise/fall delays.	- NMOS transistors are faster (lower V_{th}, higher mobility). - PMOS transistors are slower (higher V_{th}, lower mobility). - Also causes unbalanced rise/fall delays.
SF = Slow NMOS Fast PMOS	FS = Fast NMOS Slow PMOS

2 Transistor Behavior in Different Corners

Corner	NMOS Behavior	PMOS Behavior	Result
FF	Fast	Fast	Both fast
TT	Typical	Typical	Nominal
SS	Slow	Slow	Both slow
SF	Slow	Fast	Unbalanced (rise fast, fall slow)
FS	Fast	Slow	Unbalanced (rise slow, fall fast)

3 Visual Representation

SF Corner (Slow NMOS / Fast PMOS)

FS Corner (Fast NMOS / Slow PMOS)

4 Difference from SS / TT / FF

Corner	NMOS	PMOS	Main Purpose
FF	Fast	Fast	Best case (hold), minimum delay
TT	Typical	Typical	Nominal, design target
SS	Slow	Slow	Worst case (setup), maximum delay
SF	Slow	Fast	Cross-corner, unbalanced behavior
FS	Fast	Slow	Cross-corner, unbalanced behavior

5 Why Advanced Signoff Can Require SF and FS ?

- In advanced process nodes (e.g., 7nm, 5nm, 3nm), NMOS and PMOS do not always vary in the same direction.
- Due to strain engineering, different channel materials, and process-specific variations, NMOS and PMOS can have independent variations.
- SF/FS corners help capture real silicon behavior where rise and fall delays are asymmetric.
- Important for:
 - Timing signoff (Tempus)
 - SI/Noise analysis
 - Yield and reliability analysis
 - Low-power and Variation-aware design

Using only SS/TT/FF may miss timing issues caused by unbalanced NMOS/PMOS behavior.

6 Effect on Rise/Fall Delay

Corner	Rise Delay (0 → 1)	Fall Delay (1 → 0)	Observation
FF	Low	Low	Both fast
TT	Nominal	Nominal	Balanced
SS	High	High	Both slow
SF	Low	High	Fast rise, slow fall
FS	High	Low	Slow rise, fast fall

Key Takeaways:

- SF = Slow NMOS / Fast PMOS, FS = Fast NMOS / Slow PMOS.
- They model cross-corner behavior where NMOS and PMOS vary independently.
- Required in advanced nodes for accurate timing, SI, yield and reliability analysis.
- They capture asymmetric rise/fall delays which SS/TT/FF cannot.

"More Corners More Confidence in Real Silicon!"

Why is SS + Low Voltage + High Temperature generally used for Setup ?

This combination creates the slowest possible conditions, giving maximum data-path delay.

1 What is Setup Analysis ?

- Setup analysis checks whether data arrives at the flip-flop before the clock edge (launch → capture).
- We need to consider the worst case where data takes maximum time to reach the destination.
- That worst case occurs when cells are slow, supply voltage is low and temperature is high.

Setup checks the longest path delay in the design.

2 Why SS Corner Gives Slow Cells ?

SS Corner (Slow-Slow)

Higher V_{th}
Lower mobility
Thicker oxide

In SS corner, both NMOS and PMOS are slower than nominal, so the cell delay increases.

3 Why Low Voltage Slows Down ?

Supply Voltage

Voltage	Effect on Cell Delay
0.72 V	Slower (higher delay)
0.80 V	Nominal
0.88 V	Faster (lower delay)

- Lower Vdd reduces transistor drive current (I_d).
- Transistors switch slower.
- Result: higher cell delay.

4 Why High Temperature Slows Down ?

Temperature

Temperature	Effect on Cell Delay
-40°C	Faster (lower delay)
25°C	Nominal
125°C	Slower (higher delay)

- At higher temperature, carrier mobility decreases.
- Threshold voltage (V_{th}) slightly increases.
- Leakage increases and transistors switch slower.
- Result: higher cell delay.

5 Combined Effect: SS + Low V + High T

SS (Slow cells)
0.72 V (Low voltage)
125°C (High temperature)

Maximum data-path delay (slowest condition)

Hence, SS + 0.72 V + 125°C is the worst case for Setup.

6 Impact on Timing Path

With slow cells, low voltage and high temperature, the data takes maximum time to reach the capture flip-flop, which is the critical condition for setup check.

7 Setup Violation Mechanism

If data arrives after (or too close to) the clock edge (violating setup time), setup violation occurs.

Key Takeaways:

- SS corner → slower cells (higher delay)
- Low voltage (0.72 V) → reduces drive current → slower switching
- High temperature (125°C) → lower mobility → slower switching
- Together they give maximum data-path delay, so SS + 0.72 V + 125°C is used for Setup analysis.

"Slowest corner for a safe design!"

Why is FF + High Voltage + Low Temperature generally used for Hold ?

This combination creates the fastest possible conditions, giving minimum data-path delay.

1 What is Hold Analysis ?

- Hold analysis checks that data does not arrive too early at the capture flip-flop.
- If data arrives before the required hold time after the clock edge, a hold violation occurs.
- We need to consider the best case (minimum delay) where data travels as fast as possible.

Hold checks the shortest path delay in the design.

2 Why FF Corner Gives Fast Cells ?

FF Corner (Fast-Fast)

Lower V_{th}
Higher mobility
Thinner oxide
Higher drive current
Faster switching (lower delay)

In FF corner, both NMOS and PMOS are faster than nominal, so the cell delay is minimum.

3 Why High Voltage Speeds Up ?

Supply Voltage

Voltage	Effect on Cell Delay
0.72 V	Slower (higher delay)
0.80 V	Nominal
0.88 V	Faster (lower delay)

- Higher Vdd increases transistor drive current (I_d).
- Transistors switch faster.
- Result: lower cell delay.

4 Why Low Temperature Speeds Up ?

Temperature

Temperature	Effect on Cell Delay
-40°C	Faster (lower delay)
25°C	Nominal
125°C	Slower (higher delay)

- At low temperature, carrier mobility increases.
- Threshold voltage (V_{th}) slightly decreases.
- Transistors switch faster.
- Result: lower cell delay.

5 Combined Effect: FF + High V + Low T

FF (Fast cells)
0.88 V (High voltage)
-40°C (Low temperature)

Minimum data-path delay (fastest condition)

6 Effect on Data Path Delay

FF + High V + Low T gives the minimum data-path delay.

7 Hold Violation Mechanism

If the data arrives before the hold time (t_{hold}) after the clock edge, hold violation occurs.

Hence, FF + 0.88 V + -40°C is the worst case for Hold.

a hold violation occurs.

**Key Takeaways:**

- FF corner → fast cells (lower delay)
- High voltage (0.88 V) → higher drive current (Id) → faster switching
- Low temperature (-40°C) → higher mobility → faster switching
- Together they give minimum data-path delay, so FF + 0.88 V + -40°C is used for Hold analysis.

"Fast Corners
Catch Early Arrivals!"

RC CORNER**What are Setup-Worst and Hold-Worst RC Conditions?**

RC corner models the interconnect (wire) resistance and capacitance for timing analysis.

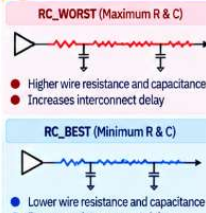
Same cells,
Different wires,
Different delays!

1 What is an RC Corner?

- RC corner models the parasitic effect of interconnect (wires).
- It defines the resistance (R) and capacitance (C) of wires used in timing analysis.
- Different RC corners represent different wire conditions (best, typical, worst).
- It is independent of the cell (PVT) corner, and both are combined for complete analysis.

2 Types of RC Corners

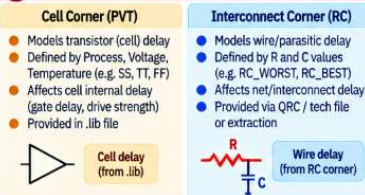
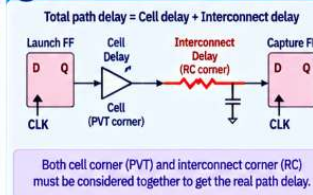
RC Corner	Wire Resistance (R)	Wire Capacitance (C)	Purpose
RC_WORST	Maximum R	Maximum C	Used for Setup analysis (worst delay)
RC_TYP	Nominal R	Nominal C	Used for general analysis (e.g. report, debug)
RC_BEST	Minimum R	Minimum C	Used for Hold analysis (fastest path)

3 Effect on Delay**4 Why Different RC for Setup and Hold?****Setup Analysis**

- Needs maximum delay
- Use RC_WORST (max R & C)
- Models the slowest interconnect
- Helps verify data arrives before clock edge

Hold Analysis

- Needs minimum delay
- Use RC_BEST (min R & C)
- Models the fastest interconnect
- Helps verify data does not arrive too early

5 Cell Corner vs Interconnect (RC) Corner**6 Combination in Timing Analysis****7 Summary Table**

Analysis Type	Cell Corner (PVT)	RC Corner	Reason
Setup	Slow (SS)	RC_WORST (max R & C)	Maximum delay (worst case)
Hold	Fast (FF)	RC_BEST (min R & C)	Minimum delay (fastest case)
Typical	Typical (TT)	RC_TYP	Nominal analysis

**Key Takeaways:**

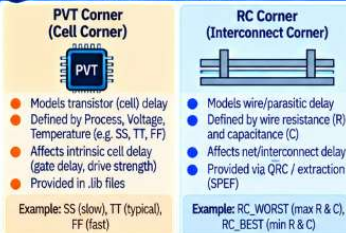
- RC_WORST → maximum R & C → used for Setup analysis (slowest interconnect).
- RC_BEST → minimum R & C → used for Hold analysis (fastest interconnect).
- Cell corner (PVT) models transistor delay; RC corner models interconnect delay.
- Both must be combined to get accurate timing results.

"Cells give delay,
Wires add more delay!"
Consider Both!"

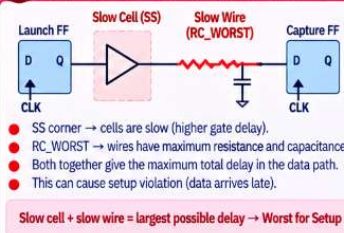
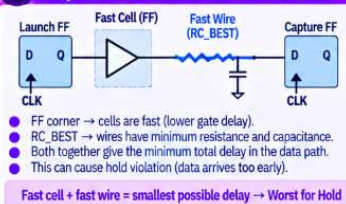
**How do PVT Corner and RC Corner Combine to Create the Real Worst-Case Scenario?**

Worst-case timing happens when both the cell (PVT) and interconnect (RC) are in the same direction.

Both Cells
and Wires Matter
for
Real Silicon!

1 PVT Corner vs RC Corner**2 Worst-Case Combinations**

Case	PVT Corner (Cell)	RC Corner (Wire)	Used For	Effect
1 (Setup Worst)	SS (Slow)	RC_WORST (Max R & C)	Setup analysis	Maximum data-path delay (slowest)
2 (Hold Worst)	FF (Fast)	RC_BEST (Min R & C)	Hold analysis	Minimum data-path delay (fastest)
3 (Typical)	TT (Typical)	RC_TYP (Nominal)	General analysis, functional	Nominal delay

3 Why Slow Cell + Slow Wire is Worst for Setup?**4 Why Fast Cell + Fast Wire is Worst for Hold?****5 Why They Must Be Modeled Separately?**

- Cells (PVT) and wires (RC) are affected by different physical factors and vary independently.
 - Cell delay depends on process, voltage and temperature (devices).
 - Wire delay depends on layout, metal layers, length, width, and parasitic extraction.
 - Worst-case for setup and hold are different combinations (SS+RC_WORST for setup, FF+RC_BEST for hold).
 - If they are not modeled separately, real silicon timing issues may be missed.
- Accurate timing signoff requires both PVT corners (cell) and RC corners (wire) to be modeled and combined properly.

6 Real-World View: N x M Combinations

PVT Corners (N)	RC Corners (M)	Total Combinations (N x M)
SS, TT, FF (3)	RC_BEST, RC_TYP, RC_WORST (3)	3 x 3 = 9

In real projects, all PVT and RC combinations are analyzed (e.g. 9 or more) to ensure the design is safe across all process, voltage, temperature and interconnect variations.

**Key Takeaways:**

- Setup worst case = Slow cell (SS) + Slow wire (RC_WORST) → maximum delay.
- Hold worst case = Fast cell (FF) + Fast wire (RC_BEST) → minimum delay.
- PVT and RC must be modeled separately because they are independent.
- Correct combination of PVT and RC corners ensures reliable timing signoff in real silicon.

"Different Corners,
Same Goal - A Reliable Chip!"

**What is a Mode in MMC?**

A mode represents a specific operating state of the chip, defining which clocks are active, what frequency they run at, which logic is active or inactive; and the related constraints.

Same Chip,
Different Modes,
Different Behavior!

1 Meaning of Operating Mode

- An operating mode is a defined set of functional and timing conditions in which the chip operates.
- Each mode represents a different use case (e.g., normal operation, low power, test mode).
- MMC analyzes timing for each mode across multiple PVT and RC corners.

One design, multiple modes,
different timing behavior!

2 Active Clocks

- A mode specifies which clocks are ON (active) and which are OFF (inactive).
- Only active clocks are used for timing analysis in that mode.

Clock	Normal Mode	Low Power Mode	Test Mode
clk_cpu	Active	Active	Inactive
clk_mem	Active	Inactive	Inactive
clk_io	Active	Inactive	Active
clk_test	Inactive	Inactive	Active

clk_cpu (active)

clk_mem (inactive)

3 Clock Frequencies

- Each active clock can have a different frequency in different modes.
- Frequency changes affect timing constraints (setup/hold).

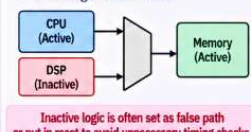
Mode	clk_cpu Frequency
Normal Mode	1.0 GHz
Low Power Mode	500 MHz
Test Mode	100 MHz

1.0 GHz (Normal Mode)

500 MHz (Low Power Mode)

4 Active / Inactive Logic

- Some blocks are enabled only in specific modes.
- Inactive blocks can be power-gated or held in reset.
- Timing analysis considers only the active logic for the mode.



5 IO Constraints

- Input/output delays, drive strengths and interface constraints can vary by mode.
- Different modes may have different external devices or operating conditions.

Set correct IO constraints for each mode.

6 False Paths

- Some paths are not functionally relevant in a particular mode.
- They are marked as false paths to exclude them from timing analysis.

Use false paths to ignore non-functional paths in a mode.

7 Multicycle Paths

- Some paths are allowed more than one clock cycle to complete.
- Use multicycle path constraints to relax timing requirements in specific modes.

Useful for low-speed paths or control paths that don't need to meet single-cycle timing.

8 Example: Modes in a SoC

Mode	Use Case	Active Clocks (Frequency)	Inactive Blocks	Special Constraints
Functional (Normal)	Regular operation	clk_cpu (1 GHz) clk_mem (800 MHz) clk_io (200 MHz)	Test logic	Nominal IO constraints
Low Power	Power saving	clk_cpu (500 MHz)	DSP, Extra IPs	Many false paths, relaxed freq.
Test	Manufacturing test	clk_test (100 MHz)	CPU, Memory	Test-specific IO, false paths to functional logic

Key Takeaways:

- A mode defines the functional and timing environment of the chip.
- It includes active clocks, clock frequencies, active/inactive logic, IO constraints, false paths and multicycle paths.
- MMC runs timing analysis for each mode across multiple PVT and RC corners.
- Proper mode definition ensures accurate and meaningful timing signoff.

"Different Modes, Different Paths, Same Reliable Silicon!"

What is Functional Mode and how is it constrained ?

Functional mode represents the normal operating condition of the chip, where all major blocks are active and the design runs at its target frequency (e.g., 1 GHz) to perform its intended functionality.

1 What is Functional Mode ?

- Functional mode is the normal operating mode where the chip performs its actual intended function.
- All functional blocks and power domains are active.
- Used to verify that the design meets performance (timing) requirements.
- Typically runs at the target frequency (e.g., 1 GHz).

It represents real-world operation of the chip.

2 Target Frequency (1 GHz)

- In functional mode, the chip operates at its rated frequency (e.g., 1 GHz).
- All active clocks (CPU, DSP, RF, etc.) run at their specified frequencies.

Clock Domain	Frequency
CPU / Core Clock	1.0 GHz
DSP Clock	500 MHz
RF Clock	250 MHz
AON Clock	100 MHz

3 Major Clocks (RF / DSP / AON)

4 All Domains Active

- In functional mode, all major power domains are ON.
- No blocks are power-gated (except test/debug if applicable).
- Includes CPU, DSP, RF, Memory, Analog (ADC/DAC), IO, etc.

Domain	Status (Functional Mode)
CPU Domain	Active
DSP Domain	Active
RF Domain	Active
AON Domain	Active
Analog (ADC/DAC)	Active
IO Domain	Active

5 Functional Paths

- Timing analysis checks all functional paths between active blocks.
- Includes paths across different domains (with proper constraints).
- Covers logic, memories, and interface logic.

Goal: Verify end-to-end functionality at target frequency.

6 Role of ADC / DAC

- In functional mode, analog interfaces like ADC and DAC are active.
- Their digital control and data paths are part of timing analysis.
- Include clock and IO constraints for these interfaces.

7 Example SDC for Functional Mode

```

# Functional Mode SDC
create_clock -name clk_cpu -period 1.000 [get_ports clk_cpu] # 1 GHz CPU clock
create_clock -name clk_dsp -period 2.000 [get_ports clk_dsp] # 500 MHz DSP clock
create_clock -name clk_rf -period 4.000 [get_ports clk_rf] # 250 MHz RF clock
create_clock -name clk_aon -period 10.000 [get_ports clk_aon] # 100 MHz AON clock

# IO constraints
set_input_delay -clock clk_cpu 0.5 [get_ports {in.*}] # Example input delay
set_output_delay -clock clk_cpu 0.5 [get_ports {out.*}] # Example output delay

# False and multicycle paths (example)
set_false_path -from [get_ports test_mode] -to [all_registers] # Test path not used
set_multicycle_path 2 -from [get_cells u_adc/*] -to [get_cells u_dac/*] # Relaxed path
  
```

Key Takeaways:

- Functional mode = normal operation at target frequency (e.g., 1 GHz).
- All domains (CPU, DSP, RF, AON, Analog, IO) are active.
- Timing analysis covers all functional paths including ADC/DAC.
- Proper SDC constraints (clocks, IO, false paths, multicycle paths) are required.
- Ensures the chip meets performance and functional requirements.

"Functional mode proves that the chip does what it is meant to do!"

What is Test/Scan Mode and how is it constrained ?

Test/Scan mode is a special operating mode used for manufacturing test, where scan chains are enabled and test data is shifted through the design at a low frequency (e.g., 100 MHz).

1 What is Test/Scan Mode ?

- Test/Scan mode is used for manufacturing test and DFT (Design for Test).
- Scan chains are enabled and test data is shifted through flip-flops.
- Functional operation is disabled during scan shift.
- Typically runs at a lower frequency (e.g., 100 MHz) for reliable testing.

Purpose: Shift test data in/out and capture test responses.

2 Key Characteristics

Mode Name	Test / Scan Mode
Typical Frequency	100 MHz
scan_en	1 (enabled)
Active Paths	Scan shift paths (SI → Q)
Functional Paths	Disabled / False paths
Used For	Manufacturing test (DFT / ATE)

3 Scan Chain Operation

Scan Shift Waveforms (Test Mode)

4 Timing Focus: SI → Q

- During scan shift, timing analysis checks the path from Scan In (SI) to Scan Flip-Flop Q.
- Ensures scan chain can operate at the test frequency (e.g., 100 MHz).
- Functional timing paths are not relevant and should be disabled.

Check timing during scan shift (SI → Q)

5 Functional Paths Disabled

- In test mode, normal functional clocks and paths are disabled or set as false paths.
- Prevents false timing violations during scan analysis.
- Only scan-related paths (scan_clk, SI → Q) are analyzed.
- Functional clocks can be turned off or constrained as false paths.

Goal: Analyze only scan paths, ignore functional paths.

6 Example Test SDC (Scan Mode)

```

# Test / Scan Mode SDC
create_clock -name scan_clk -period 10.0 [get_ports scan_clk] # Define scan clock (100 MHz)

# Set scan enable (scan_en = 1)
set_case_analysis 1 [get_ports scan_en]

# Disable functional clocks / paths
set_false_path -from [all_clocks -except scan_clk]

# (Optional) Disable functional I/O during scan
set_false_path -from [all_inputs] -to [all_outputs]

# Analyze only scan shift path (SI → Q is automatic with DFT cells)
  
```

7 Functional vs Test Mode

Item	Functional Mode	Test/Scan Mode
Frequency	1 GHz	100 MHz
scan_en	0	1
Active Paths	Functional paths	Scan shift paths (SI → Q)
Functional Paths	Enabled	Disabled (False)
Clocks	CPU/DSP/etc.	Scan clock only
Purpose	Normal operation	Manufacturing test

8 Summary

- Test/Scan mode is used for manufacturing test.
- Typically runs at 100 MHz.
- scan_en = 1 enables scan chains.
- Timing checks SI → Q during scan shift.
- Functional paths are disabled (using false paths).
- Requires test-specific SDC constraints.

Correct test mode constraints ensure clean scan timing analysis and avoid false violations.

Key Takeaways:

- Test/Scan mode = DFT mode for manufacturing test.
- Runs at lower frequency (e.g., 100 MHz).
- scan_en = 1 enables scan chains.
- Analyze only scan shift paths (SI → Q), disable functional paths.
- Use test-specific SDC with scan clock and false path constraints.

"Test mode finds defects, Constraints make it work!"

What is Boot Mode and why does it need a separate constraint mode ?

Boot mode is the initial power-on mode where a minimal set of blocks are active to bring up the chip. It needs a separate constraint mode because the active clocks, active logic and timing requirements are different from functional and test modes.

1 What is Boot Mode ?

- Boot mode is the initial power-on operating mode.
- Only essential blocks are active to initialize and configure the chip.

2 Key Characteristics

Item	Boot Mode Setting
Boot clock frequency	200 MHz

3 Block Diagram (Boot Mode)

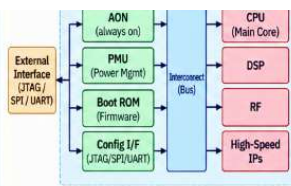
4 Clocks in Boot Mode

Clock Name	Frequency	Status in Boot Mode
clk_boot	200 MHz	Active
clk_aon	50 MHz	Active

- Typically runs at a lower frequency than functional mode (e.g., 200 MHz).
- Used to load firmware from boot ROM and configure the system.
- Needs a separate constraint mode because the active clocks, paths and timing requirements are different from functional and test modes.

Goal: Bring up the chip safely and reliably with minimum active logic.

Active blocks	AON / PMU / boot ROM (Configuration logic)
Inactive blocks	RF, DSP, High-speed IPs
Active paths	Minimal (only boot related)
Main purpose	Initialize, configure and load firmware
Interface used	Configuration interface (e.g., JTAG, SPI, UART)



clk_pmu	100 MHz	Active
clk_cpu	1 GHz	Inactive
clk_dsp	500 MHz	Inactive
clk_rf	250 MHz	Inactive

clk_boot (200 MHz) T = 5 ns

5 Why Separate Constraint Mode ?

- Different active clocks (e.g., 200 MHz boot clock instead of 1 GHz functional clock).
- Only a subset of blocks are active (AON/PMU/boot ROM).
- Many blocks (RF/DSP/etc.) are powered down or clock-gated.
- Functional paths are false/irrelevant in boot mode.
- Different timing requirements (configuration interface, boot sequence).

Using functional mode constraints can either miss violations or create false errors.

6 Example Boot-specific SDC

```
# Boot Mode SDC
create_clock -name clk_boot -period 5.000 \
[get_ports clk_boot] # 200 MHz

# AON / PMU clocks
create_clock -name clk_aon -period 20.000 \
[get_ports clk_aon] # 50 MHz
create_clock -name clk_pmu -period 10.000 \
[get_ports clk_pmu] # 100 MHz

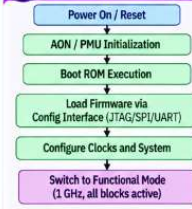
# Disable functional clocks (not used in boot)
set_false_path -from [get_clocks clk_cpu]
set_false_path -from [get_clocks clk_dsp]
set_false_path -from [get_clocks clk_rf]

# Constrain configuration interface (example)
set_input_delay 2.0 -clock clk_boot [get_ports spi.*]
set_output_delay 2.0 -clock clk_boot [get_ports spi.*]
```

7 Mode Comparison

Item	Boot Mode	Functional Mode	Test/Scan Mode
Main purpose	Initialize & configure chip	Normal operation	Manufacturing test (DFT)
Target frequency	200 MHz	1 GHz	100 MHz
Active blocks	AON/PMU/Boot ROM	All blocks (CPU, DSP, RF, etc.)	Scan chain only
Inactive blocks	RF, DSP, etc.	None (all active)	Most functional logic (scan only)
Main paths analyzed	Boot/config paths only	All functional paths	Scan shift paths (SI → Q)
Constraint file	boot_mode.sdc	func_mode.sdc	test_mode.sdc

8 Typical Boot Sequence



Key Takeaways:

- Boot mode runs at ~200 MHz with only AON/PMU/boot ROM and configuration interface active.
- RF/DSP and other high-speed clocks are off.
- Only minimal boot-related paths are analyzed.
- Requires a separate SDC (boot_mode.sdc) with appropriate clocks, false paths and IO constraints.
- Separate mode ensures accurate timing analysis and avoids false violations.

"Right constraints for the right mode lead to a reliable chip!"

Why does every operating mode require its own .SDC ?

Each mode represents a different operating condition of the chip. The clocks, active logic, timing paths and constraints change from mode to mode, so a separate .SDC is required for accurate timing analysis.

Different Modes
Different Constraints
Same Goal
A Reliable Chip!

1 Different create_clock

- Each mode has different active clocks and frequencies.
- Clock names, frequencies and generated clocks change per mode.
- Inactive clocks should not be constrained in that mode.

```
# Functional mode
create_clock -name clk_cpu -period 1.0 [get_ports clk_cpu]

# Test mode
create_clock -name scan_clk -period 10.0 [get_ports scan_clk]

# Boot mode
create_clock -name clk_boot -period 5.0 [get_ports clk_boot]
```

2 Different false paths

- Some paths are not relevant in a particular mode and must be set as false paths.
- Example: scan paths in functional mode, or functional paths in test mode.

```
# Functional mode: disable scan paths
set_false_path -from [get_ports scan_en]
set_false_path -through [get_cells *scan*]

# Test mode: disable functional paths
set_false_path -from [get_ports clk_cpu]

# Boot mode: disable RF/DSP paths
set_false_path -from [get_clocks clk_rf]
set_false_path -from [get_clocks clk_dsp]
```

3 Different IO delays

- IO timing depends on the operating condition and interface used.
- External devices, board requirements and interfaces (JTAG, SPI, etc.) change per mode.

```
# Functional mode (normal operation)
set_input_delay 2.0 -clock clk_cpu [get_ports din]
set_output_delay 2.0 -clock clk_cpu [get_ports dout]

# Test mode (ATE)
set_input_delay 5.0 -clock scan_clk [get_ports scan_in]
set_output_delay 5.0 -clock scan_clk [get_ports scan_out]

# Boot mode (configuration interface)
set_input_delay 3.0 -clock clk_boot [get_ports spi.*]
set_output_delay 3.0 -clock clk_boot [get_ports spi.*]
```

4 Different multicycle paths

- Some paths are allowed more than one clock cycle in specific modes.
- Multicycle settings depend on the functionality and clock frequency of the mode.

```
# Example: config register takes 4 cycles (set only in boot mode)
set_multicycle_path 4 -from [get_cells u_cfg*] \
-to [get_cells u_cfg*]

# Example: test logic with 2-cycle path (set only in test mode)
set_multicycle_path 2 -from [get_cells u_scan*] \
-to [get_cells u_scan*]
```

5 Different case analysis

- Mode-specific control signals must be fixed using case analysis to reflect the actual hardware state.
- This helps the tool understand which logic is active or inactive.

```
# Functional mode (scan disabled)
set_case_analysis 0 [get_ports scan_en]

# Test mode (scan enabled)
set_case_analysis 1 [get_ports scan_en]

# Boot mode (enable boot logic)
set_case_analysis 1 [get_ports boot_mode]
set_case_analysis 0 [get_ports func_mode]
```

6 Example: Constraints Vary Across Modes

Constraint Type	Functional Mode	Test/Scan Mode	Boot Mode
Clock (create_clock)	clk_cpu = 1 GHz	scan_clk = 100 MHz	clk_boot = 200 MHz
Active Clocks	CPU, DSP, etc.	Scan clock only	Boot clock + AON/PMU
False Paths	Scan paths	Functional paths	RF/DSP paths
IO Delays	2 ns (system IO)	5 ns (ATE)	3 ns (SPI/JTAG)
Multicycle Paths	As per design	Scan logic (e.g. 2 cycles)	Config paths (e.g. 4 cycles)
Case Analysis	scan_en = 0	scan_en = 1	boot_mode = 1
Main Purpose	Normal operation	Manufacturing test	Initialize & configure chip

7 Why combining all modes in one .SDC is problematic ?

- Conflicting clock definitions (same clock name, different frequency).
- Incorrect timing analysis (invalid constraints for some paths).
- Unnecessary constraints on inactive logic (reduces optimization quality).
- Hard to debug and maintain.
- May hide real timing violations or create false errors.

Each mode has a unique operating environment, so it must have its own .SDC for accurate, meaningful and reliable timing signoff.

Key Takeaways:

- Every operating mode has different clocks, false paths, IO delays, multicycle paths and case analysis.
- A separate .SDC per mode ensures accurate and realistic timing analysis.
- Combining all modes into one .SDC leads to conflicting constraints and unreliable results.
- Use MMMC with mode-specific .SDCs for a clean, correct and maintainable flow.

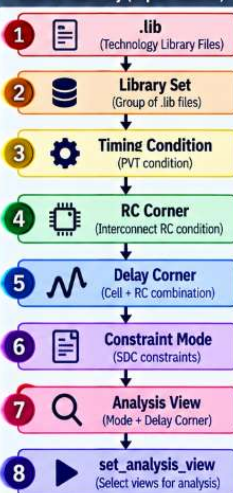
"Right constraints for the right mode lead to the right silicon!"

What is the complete hierarchy inside an MMMC file ?

An MMMC file defines the relationship between libraries, corners, modes and analysis views for timing analysis. It follows a hierarchical structure from technology libraries to the final analysis views used in STA.

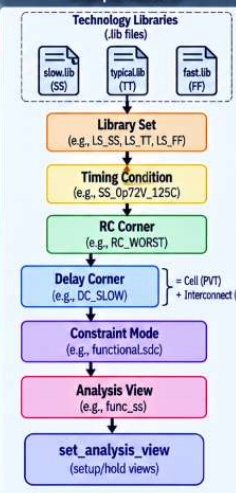
Different Blocks
Work Together
for Accurate
Timing Signoff!

MMMC Hierarchy (Top to Bottom)



Level	What it is ?	Key Details / Example
1 .lib	Standard cell library files characterized at different PVT conditions.	• .lib files for SS, TT, FF • Different voltage and temperature • Example: slow.lib, typical.lib, fast.lib
2 Library Set	A collection of .lib files grouped for a specific timing condition.	• One library set per PVT (e.g., SS_0p72V_125C) • Can include multiple .lib (e.g., std, IO, memory) • Created using create_library_set
3 Timing Condition	Defines the PVT condition (Process, Voltage, Temperature).	• Example: SS_0p72V_125C, TT_0p8V_25C, FF_0p8V_-40C • References a library set • Created using create_timing_condition
4 RC Corner	Defines the interconnect RC condition (parasitics).	• RC_WORST (max RC) or RC_BEST (min RC) • Includes TLU+ files (R, C, layer info) • Created using create_rc_corner
5 Delay Corner	Combination of a timing condition (cell) and an RC corner (interconnect).	• Defines overall delay behavior • Example: DC_SLOW = (SS_0p72V_125C + RC_WORST) • Created using create_delay_corner
6 Constraint Mode	Defines the operating mode with its SDC constraints.	• Includes clocks, I/O delays, false paths, multicycle paths • Example: functional.sdc, test.sdc, boot.sdc • Created using create_constraint_mode
7 Analysis View	A combination of a constraint mode and a delay corner.	• Defines one view for timing analysis • Example: functional_ss (functional mode + slow corner) • Created using create_analysis_view
8 set_analysis_view	Selects the analysis views to be used in STA (setup and/or hold).	• Specify setup views and hold views • Example: set_analysis_view -setup {func_ss func_tt} \ -hold {func_ff}

Example Structure



Key Takeaways:

- MMMC defines a hierarchy from technology libraries to final analysis views.
- Each level builds on the previous level.
- A delay corner = timing condition (cell) + RC corner (interconnect).
- Analysis view = constraint mode + delay corner.
- Use set_analysis_view to specify the views for setup and hold analysis.

"A Well-Defined MMMC Structure Leads to Accurate and Reliable Timing Signoff!"

What is a Library Set ?

A library set is a collection of .lib files (technology libraries) that are used together in STA for a particular PVT condition. All the libraries in a set must have the same PVT (Process, Voltage, Temperature).

Group of .lib files with same PVT = Library Set !

1 What is a Library Set ?

- A library set is a group of .lib files used together for timing analysis at a specific PVT condition.
 - It represents all the timing libraries required for the design (e.g., standard cells, SRAM, PLL, etc.) that operate at the same PVT.
 - Created using the **create_library_set** command in the MMMC file.
 - Each timing condition (e.g., SS_0p72V_125C) will reference one library set.
- One library set = Multiple .lib files with the same PVT.

2 Why Multiple .lib Files are Grouped ?

- A real design uses different types of cells and IPs, each delivered with its own .lib file.
- These .lib files are grouped so that STA can analyze the complete design (standard cells + memories + analog/mixed-signal IPs) together.
- All libraries in the set must have the same PVT (e.g., all at SS, 0.72 V, 125°C).
- Grouping ensures consistent timing, proper interactions, and accurate analysis across the design.

3 Typical Libraries in a Library Set

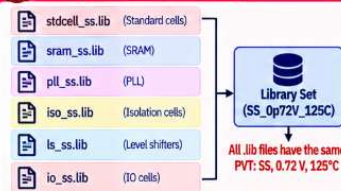
	Standard Cells	Main digital logic cells (combinational, sequential)
	SRAM / Memory	SRAM, ROM, eFuse, etc.
	PLL / Clock IP	PLL, DLL, clock generators
	Isolation Cells	Used for power gating (multi-power domain)
	Level Shifters	Voltage domain crossing
	IO Cells	Pads, ESD cells, IO buffers
	Other IPs	Analog/mixed-signal blocks, custom macros, etc.

4 Same PVT Requirement

- All .lib files in a library set must correspond to the same PVT corner (e.g., SS_0p72V_125C).
- Mixing different PVT .lib files in the same library set is not allowed and will give incorrect timing results.
- For a different PVT (e.g., TT_0p8V_25C or FF_0p8V_-40C), a separate library set is created with the corresponding .lib files.

Rule: All .lib files in a library set must have the same PVT.

5 Example: .lib Files in a Library Set



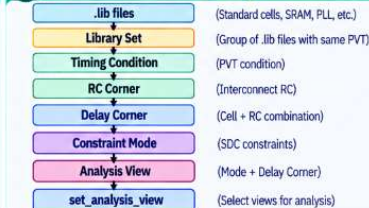
All .lib files have the same PVT: SS, 0.72 V, 125°C

6 TCL Example: create_library_set

```
# Create a library set for SS_0p72V_125C
create_library_set -name libset_ss \
-timing {
    ./lib/tech/stdcell_ss.lib
    ./lib/memory/sram_ss.lib
    ./lib/analog/pll_ss.lib
    ./lib/iso/iso_ss.lib
    ./lib/level_shifter/ls_ss.lib
    ./lib/io/io_ss.lib
}

# This library set will be used in a timing condition
create_timing_condition -name tc_ss \
-library_set libset_ss \
-operating_conditions SS_0p72V_125C
```

7 How it Fits in MMMC Hierarchy



Key Takeaways:

- A library set is a collection of .lib files used together for a specific PVT.
- It includes standard cells, SRAM, PLL, isolation cells, level shifters, IO cells, etc.
- All .lib files in a library set must have the same PVT.
- Created using **create_library_set** in the MMMC file.
- Each timing condition uses one library set.

"Right libraries, Right corner, Right results!"

Why must all libraries inside a Library Set belong to the same corner ?

All .lib files in a library set (standard cells, SRAM, PLL, ISO, level shifters, etc.) must belong to the same PVT corner so that STA uses consistent timing, power and operating conditions across the entire design.

Same Corner Same World Consistent Timing Reliable Signoff !

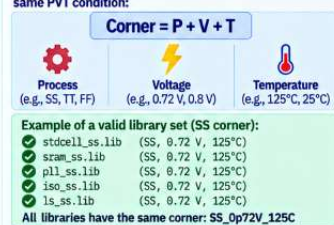
1 Libraries in a Library Set

A typical SoC library set includes multiple .lib files:

	Standard-cell .lib	Core logic cells (Combinational, Sequential)
	SRAM .lib	SRAM, ROM, eFuse, etc.
	PLL .lib	PLL, DLL, clock generators
	ISO .lib	Isolation cells (used for power gating)
	Level-shifter .lib	Voltage domain crossing
	Other .lib	IO cells, analog/mixed-signal blocks, etc.

2 What does "Same Corner" Mean ?

All .lib files in the set must be characterized at the same PVT condition:

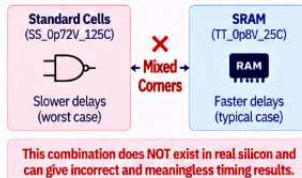


3 Why They Must Belong to the Same Corner ?

- Consistent timing:** All cells and IPs are modeled under the same PVT, so delays, setup/hold, and timing arcs are comparable.
- Accurate analysis:** STA assumes all libraries in a set operate under the same conditions.
- Correct interactions:** Ensures proper timing between standard cells and hard IPs (e.g., stdcell → SRAM, stdcell → PLL).
- Power and leakage consistency:** Power numbers are also corner dependent.
- Avoids unrealistic results:** Prevents mixing fast and slow models that do not exist in real silicon.

4 Problem: Mixing Different Corners

Example: Standard cells at SS, but SRAM at TT



5 Issues Caused by Mixing SS and TT Libraries

Issue	Description
Inconsistent timing	Delays from different corners are not comparable.
False timing violations or false passes	Paths may look faster or slower than real silicon.
Incorrect cross-IP interactions	Timing between stdcell and SRAM/PLL may be overly optimistic or pessimistic.
Wrong power estimation	Power and leakage numbers will be inconsistent. This mix does not represent any valid operating condition in the foundry data.
No real silicon correlation	Leads to inaccurate STA results and may cause silicon failures.
Signoff failure	

6 Correct vs Incorrect Example

Correct (All SS)	Incorrect (Mixed)
- stdcell_ss.lib - sram_ss.lib - pll_ss.lib - iso_ss.lib - ls_ss.lib All at SS_0p72V_125C	- stdcell_ss.lib (SS_0p72V_125C) - sram_tt.lib (TT_0p8V_25C) - pll_ss.lib (SS_0p72V_125C) - iso_ss.lib - ls_ss.lib Mixed Corners (Not Valid)

Always create separate library sets for each corner: SS, TT, FF (and their voltage/temperature variants).

Key Takeaways:

- A library set groups multiple .lib files (stdcell, SRAM, PLL, ISO, level shifters, etc.).
- All .lib files in a library set must belong to the same PVT corner.
- Mixing different corners (e.g., SS + TT) leads to incorrect and unreliable timing, power, and signoff results.
- Create separate library sets for each valid corner (e.g., SS_0p72V_125C, TT_0p8V_25C, FF_0p8V_-40C).

"Consistent corners for consistent results!"

What is a Timing Condition ?

A timing condition defines the PVT (Process, Voltage, Temperature) condition used for timing analysis. It links a library set with an operating condition (voltage, temperature, etc.) so that STA knows under which environment the libraries should be used.

Correct PVT Correct Timing Reliable Signoff !

1 What is a Timing Condition ?

- A timing condition represents a specific PVT condition for timing analysis.
- It specifies which library set to use and which operating condition to apply.
- It tells the tool: "Use these libraries at this voltage, temperature, and process condition."
- Created using the **create_timing_condition** command in the MMMC file.

Timing condition = Library set + Operating condition

2 Key Components

Component	Description
Library set	Group of .lib files (standard cells, SRAM, PLL, ISO, level shifters, etc.) for the same PVT corner.
Operating condition (opcond)	Defines the environment such as voltage, temperature, and process derates used for analysis. It is usually defined in the technology library (.lib) using operating_condition, and referenced in MMC using -opcond.
-opcond_library	Specifies which library contains the operating condition definition.
-opcond	Specifies the name of the operating condition to be used.

3 Command Syntax

```
create_timing_condition -name <timing_cond_name> \
-library_set <lib_set_name> \
-opcond_library <lib_name> \
-opcond <opcond_name>
```

Example:

```
create_timing_condition -name tc_ss_0p72V_125C \
-library_set libset_ss \
-opcond_library slow.lib \
-opcond SS_0p72V_125C
```

Use this library set (for SS corner)

Take opcond from this library

Use this operating condition

4 Relationship with .lib File



5 Visual Representation



6 Multiple Timing Conditions (Example)

Timing Condition	Library Set	Operating Condition	Purpose
tc_ss_0p72V_125C	libset_ss	SS_0p72V_125C	Worst case setup

temperature : 125;
...
} .lib file.

! If the -opcond name does not exist in the specified library, the tool will give an error.

PLL .lib
ISO .lib
Level shifter .lib
...

Voltage 0.72 V
Temperature 125 °C

Timing Condition
(e.g., tc_tt_0p8V_25C)

tc_tt_0p8V_25C	libset_tt	TT_0p8V_25C	Nominal
tc_ff_0p88V_40C	libset_ff	FF_0p88V_40C	Best case hold

Key Takeaways:

- A timing condition defines the PVT environment for timing analysis.
- It links a library set with an operating condition using -opcond_library and -opcond.
- The -opcond must exist in the specified .lib file.
- Different PVT corners (SS, TT, FF, etc.) require separate timing conditions.
- Correct timing conditions ensure accurate and reliable timing analysis.

"Right PVT, Right Libraries, Right Timing!"

What is an RC Corner ?

An **RC corner** defines the resistance (R) and capacitance (C) values used for interconnect modeling during static timing analysis (STA). It represents a specific combination of process, voltage and temperature (PVT) conditions, together with the appropriate RC values to analyze the worst-case delay and timing behavior of the design.

RC Corner =
R + C values +
PVT condition
(for timing analysis)

1 create_rc_corner

- Command used in STA tools (e.g., PrimeTime) to define an RC corner.
- Specifies the resistance and capacitance values to be used for timing analysis.

```
create_rc_corner rc_typ \
-qx_tech {qrc_tech_file} \
-resistance {res_table} \
-capacitance {cap_table} \
-name {rc_type}
```

- The QRC technology file contains the detailed RC tables (R and C values).

2 QRC Technology

- QRC (Quasi-Static RC) is a technology for extracting accurate interconnect resistance and capacitance values.
- It provides RC tables based on the foundry's process and metal stack.

QRC Technology → R Tables, C Tables

- These tables are used to create RC corners in STA.

3 Wire Resistance (R)

- Caused by the metal's inherent resistivity.
- Depends on metal type, width, thickness and temperature.

Metal Wire

$$R = \rho \times \frac{L}{A}$$

- Higher R → more delay (especially for long wires).

4 Wire Capacitance (C)

- Caused by the electric field between conductors and to substrate.
- Depends on spacing, width, dielectric and metal type.

Metal 1, Metal 2, Coupling Capacitance (between wires), Ground Capacitance (to substrate)

Higher C → more delay and loading.

5 Pre-route RC

- Uses estimated RC values before detailed routing.
- Based on technology defaults or early extraction models.

Synthesis Output → Pre-route RC → STA

- Used for early timing estimation and optimization.

6 Post-route RC

- Uses actual extracted RC values after routing (from tools like QRC).
- More accurate as it considers real interconnect geometry and layout.

Extracted RC (Post-route)

More accurate → realistic timing → better closure.

7 Clock RC

- Special RC values for clock nets.
- Accounts for clock tree structure, wide metals and shielding.

CLK → Clock RC (R + C)

- Important for accurate clock skew, insertion delay and jitter analysis.

8 Cross-coupling Capacitance

- Caused by parallel switching nets.
- Creates noise and affects delay.
- Modeled in advanced extraction (QRC).

Net A, Net B, C_c

- Significant for high-speed designs and aggressive routing.

Key Takeaways:

- An **RC corner** defines R and C values for timing analysis.
- It is created using **create_rc_corner** with **QRC technology**.
- Includes pre-route and post-route RC, with special handling for **clock nets**.
- Accounts for wire resistance, wire capacitance, cross-coupling capacitance and PVT conditions.

"Accurate RC data = Accurate Timing = Better Performance"

What are RC_WORST, RC_BEST and RC_TYP ?

RC corners represent different combinations of interconnect resistance (R) and capacitance (C) values used for timing analysis. They are defined using the QRC technology file and can be specified for both pre-route and post-route analysis.

Different RC, Different Realities, Better Timing Signoff!

1 What are RC_WORST, RC_BEST and RC_TYP ?

Corner	Wire Resistance (R)	Wire Capacitance (C)
RC_WORST	High	High
RC_TYP	Nominal	Nominal
RC_BEST	Low	Low

Higher R and C → Larger delay
Lower R and C → Smaller delay

2 Typical R and C Values (Illustration)

R (Ω), C (F)

3 Using -qrc_tech

- The -qrc_tech option specifies the QRC technology file used to define the RC extraction rules.
- It contains the process-specific parasitic models (R and C) for different layers, widths, spacing, and corners.
- Different QRC tech files can provide RC values for best, worst and typical corners.

```
create_rc_corner rc_worst \
-qrc_tech qrc_tech_file \
-res 1.2 -cap 1.3
```

4 Pre-route vs Post-route Options

Pre-route RC (Estimated)

- Used before routing.
- Uses technology-based models.
- Specified using pre_route_* options.

```
create_rc_corner rc_typ \
-qrc_tech qrc_tech_file \
-pre_route_res 1.0 \
-pre_route_cap 1.0
```

Post-route RC (Extracted)

- Used after routing.
- Uses extracted RC from the routed design.
- Specified using post_route_* options.

```
create_rc_corner rc_worst \
-qrc_tech qrc_tech_file \
-post_route_res 1.2 \
-post_route_cap 1.3
```

5 Example: Define RC Corners

```
create_rc_corner rc_worst \
-qrc_tech tsmc7_qrc \
-pre_route_res 1.2 \
-pre_route_cap 1.3 \
-post_route_res 1.2 \
-post_route_cap 1.3

create_rc_corner rc_best \
-qrc_tech tsmc7_qrc \
-pre_route_res 0.8 \
-pre_route_cap 0.8 \
-post_route_res 0.8 \
-post_route_cap 0.8

create_rc_corner rc_typ \
-qrc_tech tsmc7_qrc \
-pre_route_res 1.0 \
-pre_route_cap 1.0 \
-post_route_res 1.0 \
-post_route_cap 1.0
```

6 Why Setup and Hold Use Different RC Conditions ?

Setup Analysis (Max Delay)

- Uses RC_WORST (high R, high C).
- Pessimistic condition → ensures design meets timing even in the slowest case.
- Models maximum delay on data paths.

Hold Analysis (Min Delay)

- Uses RC_BEST (low R, low C).
- Optimistic condition → captures minimum delay (fastest case).
- Important to detect hold violations.

Clock, Data (RC_WORST), Data (RC_BEST)

Slower (larger delay), Faster (smaller delay)

! Using the same RC for both setup and hold can either:
• Miss real hold violations
• or be overly pessimistic, reducing performance.
Hence, different RC conditions are used.

Key Takeaways:

- RC_WORST = highest R and C (used for setup).
- RC_BEST = lowest R and C (used for hold).
- RC_TYP = nominal values (used for general analysis/correlation).
- Use -qrc_tech to specify the QRC technology file.
- Use pre_route_* for estimated RC and post_route_* for extracted RC.
- Setup and hold use different RC conditions because they analyze maximum and minimum delay scenarios.

Correct RC Corners = Reliable Timing Signoff!

What do pre_route_res/cap and post_route_res/cap actually do ?

These options are multipliers used in **create_rc_corner** to model interconnect resistance (R) and capacitance (C) for pre-route estimation and post-route extracted timing analysis.

They scale the R and C values to make the RC corner more pessimistic or optimistic, which directly impacts timing results.

1 What are these options ?

- pre_route_res** : Multiplier for wire resistance before routing.
- pre_route_cap** : Multiplier for wire capacitance before routing.
- post_route_res** : Multiplier for extracted wire resistance after routing (from SPEF).
- post_route_cap** : Multiplier for extracted

2 Where do they apply ?

Pre-route (Estimation)
Used during synthesis, early place and pre-CTS stages.
Design (placed, not routed) → RC Estimation (using technology and design rules)

Post-route (Extracted)
Used after routing, with actual extracted parasitics (SPEF).
Routed Design → Extracted RC (SPEF file)

3 How are they used ?

```
create_rc_corner rc_typ \
-qrc_tech qrc_tech_file \
-pre_route_res 1.0 \
-pre_route_cap 1.0 \
-post_route_res 1.0 \
-post_route_cap 1.0
```

4 Meaning of Common Values

Value	Meaning	Effect on RC	Use Case
1.0	No scaling (Original value)	As estimated by QRC	Typical case
0.9	10% lower than original	Less R/C (more optimistic)	Best corner (faster timing)

wire capacitance after routing (from SPEF).

They are used in the create_rc_corner command in the MMMC file.

Apply multipliers (pre_route_res/cap)

Apply multipliers (post_route_res/cap)

Each value is a multiplier applied to the R and C values estimated or extracted by the QRC tool.

Multiplier	1.1	10% higher than original	More R/C (more pessimistic)	Worst corner (slower timing)
------------	-----	--------------------------	-----------------------------	------------------------------

5 Effect on RC Values (Example)

Option	Original R/C (from QRC)	Multiplier	Final R/C (Used in STA)
pre_route_res	100 Ω	1.0	100 Ω
pre_route_res	100 Ω	0.9	90 Ω
pre_route_res	100 Ω	1.1	110 Ω
post_route_cap	20 fF	1.0	20 fF
post_route_cap	20 fF	0.9	18 fF
post_route_cap	20 fF	1.1	22 fF

6 Timing Impact

- Higher R/C (e.g., 1.1) → larger delay → more pessimistic timing (harder to meet setup).
- Lower R/C (e.g., 0.9) → smaller delay → more optimistic timing (easier to meet setup).
- Pre-route multipliers affect early timing estimation accuracy.
- Post-route multipliers adjust the extracted SPEF values to model on-chip variation, extraction depth or signoff margin.

7 Visual Representation

Pre-route RC (Estimation)
Technology-based estimation + multiplier

R (estimated) = $\times \text{pre_route_res}$

C (estimated) = $\times \text{pre_route_cap}$

Used when there is no actual routing. Estimation is based on QRC technology, wire models and design rules.

Post-route RC (Extracted)
SPEF-based RC + multiplier

R (from SPEF) = $\times \text{post_route_res}$

C (from SPEF) = $\times \text{post_route_cap}$

Used after routing with actual extracted parasitics from SPEF. Multipliers add margin or model variation.

Key Takeaways:

- pre_route_res/cap and post_route_res/cap are multipliers for R and C values.
- Pre-route uses estimated RC, post-route uses extracted SPEF.
- 1.0 means no scaling, 0.9 means 10% lower, 1.1 means 10% higher.
- They directly impact timing (higher RC → slower, lower RC → faster).
- Use appropriate values for best, typical and worst corners.

Right RC Modeling Leads to Accurate Timing and a Reliable Signoff!

What is a Delay Corner?

A Delay Corner represents a specific timing analysis condition, created by combining a Timing Condition (which defines the library/operating condition) with an RC Corner (which defines the interconnect parasitics). It is used in MMMC to model different delay scenarios like Setup, Hold and Typical.

1 Definition

- A Delay Corner is created using the command create_delay_corner.
- It combines a Timing Condition and an RC Corner.
- It defines the overall delay environment for STA.
- Used in MMMC to run analysis for setup, hold, typical or any custom scenario.

```
create_delay_corner DC_NAME \
-timing_condition TC_NAME \
-rc_corner RC_NAME
```

2 How it is Formed?

Timing Condition (TC) + RC Corner (RC) = Delay Corner (DC)

TC + RC = Complete delay environment for timing analysis

3 Common Delay Corners

Delay Corner	Purpose	Typical Composition	Used For
DC_SETUP	Worst case (max delay)	TC_SS + RC_WORST (e.g. SS, 0.72V, 125°C)	Setup analysis (timing paths must meet max delay)
DC_HOLD	Best case (min delay)	TC_FF + RC_BEST (e.g. FF, 1.08V, -40°C)	Hold analysis (timing paths must not be too fast)
DC_TYP	Typical case (nominal delay)	TC_TT + RC_TYP (e.g. TT, 0.9V, 25°C)	Debug, correlation, QoR analysis
DC_DSP	DSP-specific analysis	TC_DSP + RC_DSP (custom PVT / RC)	For DSP blocks (which may have unique PVT/RC requirements)

4 Example Commands

```
# Setup Corner (Worst case)
create_delay_corner DC_SETUP \
-timing_condition TC_SS \
-rc_corner RC_WORST

# Hold Corner (Best case)
create_delay_corner DC_HOLD \
-timing_condition TC_FF \
-rc_corner RC_BEST

# Typical Corner
create_delay_corner DC_TYP \
-timing_condition TC_TT \
-rc_corner RC_TYP

# DSP Specific Corner
create_delay_corner DC_DSP \
-timing_condition TC_DSP \
-rc_corner RC_DSP
```

5 Illustration: Delay Corner in MMMC Flow

Library Set (lib files) → Timing Condition (TC) (PVT, OpCond) + RC Corner (RC) (QRC, R/C values) → Delay Corner (DC) (TC + RC) → Constraint Mode (SDC) → Analysis View (DC + Constraint Mode)

Delay Corner defines the delay environment for timing analysis.

Each analysis view uses one delay corner and one constraint mode.

6 Why it is Important?

- Captures the real silicon behavior under different process, voltage, temperature and RC conditions.
- Ensures accurate setup and hold analysis.
- Helps model special blocks (e.g., DSP) with their own PVT/RC requirements.
- Used in signoff to verify design across all required corners.

7 Key Takeaways

- Delay Corner = Timing Condition + RC Corner
- DC_SETUP → worst case (max delay)
- DC_HOLD → best case (min delay)
- DC_TYP → nominal case
- DSP may need a separate delay corner
- Used in MMMC for accurate and complete STA

Multiple Corners Stronger Designs!

"A Delay Corner bridges the gap between technology, interconnect and timing analysis."

MMMC = More Coverage = More Confidence!

Right Corners. Right Timing. Right Silicon!

What is a Constraint Mode?

A Constraint Mode represents one operating mode of the design and contains one or more SDC files that define the timing constraints for that mode.

1 Definition

- A Constraint Mode is created using the command create_constraint_mode.
- It represents one operating mode of the design (e.g., functional, low power, test, boot).
- It contains one or more SDC files that define the timing constraints for that mode.
- Each mode can have different clocks, I/O delays, false paths, multicycle paths, etc.
- Constraint modes are later combined with delay corners to create analysis views in MMMC.

2 Command Syntax

```
create_constraint_mode MODE_NAME \
-sdc_files {file1.sdc file2.sdc ...}
```

Example:

```
create_constraint_mode FUNC \
-sdc_files {func_clk.sdc \
func_io.sdc}
```

You can specify one or more SDC files for a constraint mode.

3 Common Constraint Modes

Constraint Mode	Purpose	Typical SDC File(s)	Key Characteristics
Functional (FUNC)	Normal functional operation	func.sdc	- Active clocks - Normal I/O delays - Functional timing paths
Low Power (LP)	Low power operation (e.g., clock gating, power down blocks)	low_power.sdc	- Reduced/disabled clocks - Additional false paths - Power intent constraints
Test (TEST)	During manufacturing test (scan mode)	test.sdc	- Scan clocks - Test-specific constraints - Functional paths may be false
Boot (BOOT)	Initial startup sequence	boot.sdc	- Limited clocks - Special initialization paths - Different I/O constraints

4 Relationship Between Mode and Constraints

Operating Mode (e.g., Functional, Low Power, Test, Boot) → SDC File 1 (create_clock, I/O delays, etc.) → SDC File 2 (false paths, multicycle paths, etc.) → SDC File 3 (additional constraints) → Constraint Mode (e.g., FUNC)

A constraint mode = One operating mode + One or more SDC files that define all timing constraints for that mode.

5 How it Fits in MMMC Flow

Library Set (lib files) + Constraint Mode (.sdc files) → Analysis View (Delay Corner + Constraint Mode) → set_analysis_view (List of analysis views)

6 Why Different Modes Are Needed?

- Different operating modes have different timing requirements.
- Clocks, I/O delays, false paths, and multicycle paths can change per mode.
- Separating modes makes constraints easier to manage and maintain.
- Ensures accurate STA across all real use cases (functional, low power, test, boot, etc.).

7 Key Takeaways

- create_constraint_mode defines one operating mode.
- Can include one or more SDC files.
- Common modes: Functional, Low Power, Test, Boot.
- Each mode has its own set of constraints.
- Combined with delay corners to form analysis views in MMMC.

Key Message:

"Different operating modes need different constraints, and each mode is defined using a Constraint Mode."

Right Mode. Right Constraints. Reliable Signoff!

What is an Analysis View?

An Analysis View represents one complete STA scenario, created by combining

Right Scenario. Right Analysis. Right Signoff!

Constraint Mode (defines the functional mode and constraints) and a **Delay Corner** (defines timing environment).

1 Definition

- An Analysis View is created using the command `create_analysis_view`.
- It combines a Constraint Mode and a Delay Corner.
- It represents one complete STA scenario with libraries, RC, and constraints.
- Used in MMMC to run setup, hold, typical or any custom analysis.

```
create_analysis_view VIEW_NAME \
-constraint_mode MODE_NAME \
-delay_corner DC_NAME
```

2 How it is Formed ?

Constraint Mode (MODE)

- One operating mode
- One or more SDC files
- Defines functional intent (e.g., functional, low power, test, boot)

Delay Corner (DC)

- Timing condition (library + opcond)
- RC corner (RC)
- Defines timing environment (e.g., DC_SETUP, DC_HOLD)

Analysis View (AV)
= Complete STA scenario (MODE + DC)

3 Types of Analysis Views

View Type	Constraint Mode (Example)	Delay Corner (Example)	Purpose
Setup View	Functional (func_mode)	DC_SETUP (SS + RC_WORST)	Checks maximum delay (paths must meet setup requirements).
Hold View	Functional (func_mode)	DC_HOLD (FF + RC_BEST)	Checks minimum delay (paths must meet hold requirements).
Typical View	Functional (func_mode)	DC_TYP (TT + RC_TYP)	Used for debugging, correlation and QoR analysis (not for signoff).
Low Power View	Low power (lp_mode)	DC_SETUP/HOLD	Analyzes design in low power mode (different constraints).

4 Example Commands

```
# Setup analysis view
create_analysis_view AV_SETUP \
-constraint_mode FUNC_MODE \
-delay_corner DC_SETUP

# Hold analysis view
create_analysis_view AV_HOLD \
-constraint_mode FUNC_MODE \
-delay_corner DC_HOLD

# Typical analysis view
create_analysis_view AV_TYP \
-constraint_mode FUNC_MODE \
-delay_corner DC_TYP
```

5 Illustration: Analysis View in MMMC

Each analysis view represents a specific combination of mode and corner, because timing results depend on both the design intent (constraints) and the environment (libraries + RC).

STA Tools (set_analysis_view AV_SETUP) → Runs setup analysis for this scenario

6 Why Each View Represents a Specific Combination ?

- Different modes have different constraints (clocks, I/O, false paths, etc.).
- Different delay corners have different library, voltage, temperature and RC conditions.
- Timing results depend on both the design intent (mode) and the environment (corner).
- Separating them into views allows independent analysis of setup, hold, typical, low power, test, etc.
- Ensures accurate and complete signoff across all required scenarios.

7 Key Takeaways

- Analysis View = Constraint Mode + Delay Corner.
- Each view is one complete STA scenario.
- Common views: Setup, Hold, Typical (and custom views like Low Power).
- Used in MMMC for accurate and efficient multi-scenario analysis.
- Every view is unique because it represents a specific mode and corner combination.

Key Message: "An Analysis View captures the full timing picture for a specific mode and environment."

More Views Stronger Signoff!

How do you build Functional Setup/Hold, Low-Power and Test Analysis Views ?

Create constraint modes, delay corners and then combine them using `create_analysis_view` to build the required views.

1 Create Constraint Modes

Define one constraint mode for each operating mode and attach the required SDC file(s).

```
# Functional mode
create_constraint_mode FUNC \
-sdc_files {func.sdc}

# Low power mode
create_constraint_mode LP \
-sdc_files {low_power.sdc}

# Test mode
create_constraint_mode TEST \
-sdc_files {test.sdc}

# Boot mode
create_constraint_mode BOOT \
-sdc_files {boot.sdc}
```

Each constraint mode represents one operating mode with its own set of constraints.

2 Create Delay Corners

Define delay corners for setup, hold and typical (you can use your library sets, RC corners and timing conditions).

```
# Setup delay corner
create_delay_corner DC_SETUP \
-timing_condition TC_SS \
-rc_corner RC_WORST

# Hold delay corner
create_delay_corner DC_HOLD \
-timing_condition TC_FF \
-rc_corner RC_BEST

# Typical delay corner
create_delay_corner DC_TYP \
-timing_condition TC_TT \
-rc_corner RC_TYP
```

Each delay corner represents a specific PVT + RC environment for timing analysis.

3 Create Analysis Views

Combine the constraint mode and delay corner to create complete analysis views for different scenarios.

```
# Functional Setup View
create_analysis_view AV_FUNC_SETUP \
-constraint_mode FUNC \
-delay_corner DC_SETUP

# Functional Hold View
create_analysis_view AV_FUNC_HOLD \
-constraint_mode FUNC \
-delay_corner DC_HOLD

# Low-Power Setup View
create_analysis_view AV_LP_SETUP \
-constraint_mode LP \
-delay_corner DC_SETUP

# Low-Power Hold View
create_analysis_view AV_LP_HOLD \
-constraint_mode LP \
-delay_corner DC_HOLD

# Test Setup View
create_analysis_view AV_TEST_SETUP \
-constraint_mode TEST \
-delay_corner DC_SETUP

# Boot Setup View
create_analysis_view AV_BOOT_SETUP \
-constraint_mode BOOT \
-delay_corner DC_SETUP

# Functional Typical View
create_analysis_view AV_FUNC_TYP \
-constraint_mode FUNC \
-delay_corner DC_TYP
```

Each analysis view = One complete STA scenario (Constraint Mode + Delay Corner).

4 Summary of Created Analysis Views

Analysis View	Constraint Mode (Mode)	Delay Corner (DC)	Purpose
AV_FUNC_SETUP	FUNC (Functional)	DC_SETUP	Functional setup analysis
AV_FUNC_HOLD	FUNC (Functional)	DC_HOLD	Functional hold analysis
AV_LP_SETUP	LP (Low Power)	DC_SETUP	Low-power setup analysis
AV_LP_HOLD	LP (Low Power)	DC_HOLD	Low-power hold analysis
AV_TEST_SETUP	TEST (Test)	DC_SETUP	Test mode setup analysis
AV_BOOT_SETUP	BOOT (Boot)	DC_SETUP	Boot mode setup analysis
AV_FUNC_TYP	FUNC (Functional)	DC_TYP	Functional typical analysis (debug / correlation)

5 Overall Flow

Constraint Modes (MODE) + Delay Corners (DC) = Analysis Views (AV)

Analysis Views (AV): AV_FUNC_SETUP, AV_FUNC_HOLD, AV_LP_SETUP, AV_LP_HOLD, AV_TEST_SETUP, AV_BOOT_SETUP, AV_FUNC_TYP

Run STA (setup, hold, closure, signoff)

6 Key Takeaways

- A constraint mode defines the operating mode with one or more SDC files.
- A delay corner defines the PVT and RC environment.
- An analysis view = Constraint Mode + Delay Corner (one complete STA scenario).
- Create separate views for Functional, Low-Power, Test, Boot, etc.
- Use the appropriate views in MMMC to run accurate and complete STA.

Right views. Accurate timing. Reliable silicon!

What is set_analysis_view, and which views are used for signoff ?

The `set_analysis_view` command selects the active analysis views (setup and hold) that the tool (Tempus/Innovus) will use for timing analysis, optimization, and signoff checks.

1 What is set_analysis_view ?

- `set_analysis_view` specifies which analysis views are active for timing analysis.
- It tells the tool which view to use for setup checks and which view to use for hold checks.
- Each view = Constraint Mode + Delay Corner (one complete STA scenario).
- Used in Tempus (signoff STA) and Innovus (in-implementation STA).
- You can switch active views at any time during the flow.

Command Syntax:

```
set_analysis_view \
-setup <setup_view_name> \
-hold <hold_view_name>
```

2 Example Usage

```
# Set functional setup and hold views
set_analysis_view \
-setup AV_FUNC_SETUP \
-hold AV_FUNC_HOLD

# Set low-power setup and hold views
set_analysis_view \
-setup AV_LP_SETUP \
-hold AV_LP_HOLD

# Set test setup view (test usually checks setup only)
set_analysis_view \
-setup AV_TEST_SETUP \
-hold AV_TEST_SETUP ;# (if hold not required)
```

The specified views become the active views used by the tool for analysis and optimization.

3 Which Views are Used for Signoff ?

Stage	Setup View	Hold View	Purpose
Functional Signoff	AV_FUNC_SETUP	AV_FUNC_HOLD	Final signoff for normal functional operation (mandatory)
Low-Power Signoff	AV_LP_SETUP	AV_LP_HOLD	Signoff for low-power mode (e.g., retention, power gating)
Test Signoff	AV_TEST_SETUP	(optional) AV_TEST_HOLD	Ensure test mode timing (scan, ATPG)
Boot Signoff	AV_BOOT_SETUP	(optional) AV_BOOT_HOLD	Startup/initialization sequence timing
Typical View (Non-signoff)	AV_FUNC_TYP	(optional)	Used for debugging, correlation and QoR analysis (not for final signoff)

4 Why Setup and Hold Need Different Corners ?

Setup (Worst case - Slow)

Slow corner (e.g., SS, low V, high T) and high RC (RC_WORST) → paths are slower → check maximum delay → ensures data arrives before

Hold (Best case - Fast)

Fast corner (e.g., FF, high V, low T) and low RC (RC_BEST) → paths are faster → check minimum delay → ensures data does not arrive

5 How Tempus/Innovus Use the Selected Views ?

- The selected setup and hold views become the **active** views for all STA commands (report_timing, report_qor, etc.).
- Timing optimization uses the active views to fix setup or hold violations.
- Tempus performs signoff STA (more advanced engines, SI, noise, DCV, AOCV, etc.) using the active views.
- Innovus uses the same views during implementation for

6 Key Takeaways

- `set_analysis_view` selects the active setup and hold views.
- Use -setup for the setup view and -hold for the hold view.
- Signoff typically uses functional and low-power views (at minimum).
- Setup and hold use different corners because worst and best delays occur under different conditions.

Right Views Right Corners Accurate Timing Reliable Signoff!

capture clock edge too early

💡 Setup and hold use different corners because the worst delay and best delay occur under different PVT and RC conditions.

timing analysis and optimization (place, CTS, route).

✓ You can change the active views to analyze different modes (functional, low-power, test, etc.).

⚙️ **Same analysis views across Innovus and Tempus** ensures consistent results from implementation to signoff.

5 Tempus/Innovus use the active views for timing analysis, optimization and signoff checks.

6 Keep view definitions consistent across the flow for reliable signoff.

🎯 **Right Analysis Views → Right Timing → Successful Signoff!**

How does .MMMC interact with .LIB, .SDC, .SPEF, .UPF and .QRCtech?

The .mmmc file does not replace these files, it organizes and connects them to create complete STA analysis views across multiple modes, corners and scenarios.

1 .MMMC + .LIB

- .lib contains timing, power and cell characterization data for a specific PVT.
- .mmmc uses .lib to create a **Library Set**.
- Different corners use different lib files (e.g., SS, TT, FF).

2 .MMMC + .SDC

- .sdc contains design constraints (clocks, I/O delays, false paths, etc.).
- .mmmc groups one or more .sdc files into a **Constraint Mode**.
- Each mode (functional, low-power, test, boot) can use different .sdc files.

3 .MMMC + .SPEF

- .speg contains extracted parasitics (post-route RC).
- .mmmc can reference .speg for post-route timing (via RC corner).
- Pre-route uses estimated RC (no SPEF), post-route uses extracted SPEF.

4 .MMMC + .UPF

- .upf defines power intent (power domains, voltage levels, power states).
- .mmmc uses .upf to understand multi-voltage/multi-domain scenarios.
- Different modes (e.g., low-power) can use different power states.

5 .MMMC + .QRCtech

- .qrcTech defines process technology for RC extraction (layer properties, resistance, capacitance, etc.).
- .mmmc uses .qrcTech to create RC corners.
- Defines how wire resistance and capacitance are estimated or extracted.

Complete STA Ecosystem (How .MMMC connects everything)

```

graph TD
    LIB[.LIB  
(Timing, Power, PVT data)] --> LS[Library Set  
(create_library_set)]
    SDC[.SDC  
(Constraints)] --> CM[Constraint Mode  
(create_constraint_mode)]
    SPEF[.SPEF  
(Extracted RC)] --> RC[RC Corner  
(create_rc_corner)]
    UPF[.UPF  
(Power Intent)] --> AV[Analysis View  
(create_analysis_view)]
    QRC[.QRCtech] --> RC
    LS --> CM
    CM --> DC[Delay Corner  
(create_delay_corner)]
    RC --> DC
    DC --> AV
    AV --> SAV[set_analysis_view  
(Select active views for STA)]
  
```

Flow Summary

- Read .lib files and create Library Set.
- Read .sdc files and create Constraint Modes.
- Read .qrcTech and create RC Corners.
- Combine Library Set + RC Corner to create Delay Corners.
- Combine Constraint Mode + Delay Corner to create Analysis Views.
- .speg is used for post-route timing (as part of RC).
- .upf provides voltage/domain information for multi-power analysis.
- Use **set_analysis_view** to select the active setup/hold (and other) views for STA.

Example: Key MMMC Commands

```

# 1. Create library set (from .lib files)
create_library_set LS \
-timing {ss.lib tt.lib ff.lib}

# 2. Create constraint mode (from .sdc files)
create_constraint_mode FUNC \
-sdc_files {func.sdc}

# 3. Create RC corner (using .qrcTech)
create_rc_corner RC_WORST -qrc_tech my_qrcTech ...

# 4. Create delay corner
create_delay_corner DC_SETUP \
-library_set LS -rc_corner RC_WORST

# 5. Create analysis view
create_analysis_view AV_FUNC_SETUP \
-constraint_mode FUNC -delay_corner DC_SETUP

# 6. Select active views
set_analysis_view -setup AV_FUNC_SETUP -hold AV_FUNC_HOLD
  
```

Key Message:

"MMMC is the glue that connects all the pieces (.LIB, .SDC, .SPEF, .UPF, .QRCtech) to create complete and accurate STA scenarios."

✓ Right Files. Right Corners.
Right Views. **Reliable Signoff!**

How do you load, verify and debug a complete MMMC setup in Innovus/Tempus?

Follow a systematic flow to read the MMMC file, verify all views and settings, run timing checks, and debug any issues.

1 Load MMMC Setup

Load the MMMC file which creates library sets, constraint modes, RC corners, delay corners and analysis views.

```

innovus> read_mmc mimo_mmc.tcl
# or in Tempus
% read_mmc mimo_mmc.tcl
  
```

✓ Check for any errors or warnings during MMMC read. All referenced files (.lib, .sdc, .qrcTech, etc.) should be accessible.

2 Verify Setup

Use report commands to verify that everything is created correctly.

```

report_analysis_views
# List all analysis views

report_library_sets
# List library sets and libraries

report_constraint_modes
# List constraint modes and SDC files

report_delay_corners
# List delay corners (timing + RC)

report_rc_corners
# List RC corners

report_timing_conditions
# List timing conditions (PVT)
  
```

3 Run STA Checks

Run basic timing checks to validate the setup.

```

check_timing -verbose
# Check for common issues

report_timing_summary
# Overall timing summary for all views

report_timing -view AV_FUNC_SETUP
# Detailed timing report for a specific view

report_timing -view AV_FUNC_HOLD
# Hold timing report

report_timing -view AV_LP_SETUP
# Low-power setup view timing

report_timing -view AV_TEST_SETUP
# Test mode timing
  
```

4 Expected Correct Setup

When everything is correct, you should see:

- ✓ All analysis views are listed.
- ✓ Library sets show correct .lib files.
- ✓ Constraint modes show correct .sdc files.
- ✓ RC corners are linked to correct .qrcTech.
- ✓ Delay corners show timing condition + RC corner.
- ✓ No major errors in check_timing -verbose.
- ✓ Timing reports show meaningful data (no 'all paths unconstrained').
- ✓ Setup and hold views use correct corners.
- ✓ SPEF is used for post-route (if available).

5 Common Issues and How to Debug

#	Issue	Symptoms / Error Messages	Possible Cause	How to Debug / Fix
1	Missing .lib	Error: Cannot open library file Library not found	Wrong path or missing file	Check path in MMMC, verify file exists Use report_library_sets
2	Wrong .SDC	Clocks not found, many unconstrained paths	Incorrect SDC file or wrong mode	Check SDC path in constraint mode Use report_constraint_modes
3	Missing analysis view	Error: analysis view not found	View not created in MMMC	Check report_analysis_views Verify create_analysis_view commands
4	Incorrect setup/hold corner	Setup violations too large/small Hold violations abnormal	Wrong PVT or RC corner used	Check report_delay_corners Verify setup uses worst, hold uses best
5	SPEF/RC-corner mismatch	RC corner mismatch warnings Inconsistent parasitics	SPEF extracted with different corners	Check SPEF header, verify RC corner Use report_rc_corners
6	Wrong voltage library	Power/voltage mismatch warnings	Using library for wrong VDD	Verify library set and voltage in MMMC Check UPF and voltage mapping
7	Unconstrained paths	Warning: X paths unconstrained	Missing constraints in SDC	Check report_timing -unconstrained Add proper clock/I/O constraints
8	Incorrect mode constraints	Signals/clocks missing in specific mode	Wrong SDC in constraint mode	Verify create_constraint_mode mapping Check mode-specific SDC content
9	Inconsistent library corners	Corner mismatch warnings Timing results look incorrect	Library, RC and PVT not aligned	Check library set, RC corner and timing condition Ensure consistent process/voltage/temperature

6 Example Console Output (Simplified)

```

innovus> read_mmc mimo_mmc.tcl
Reading MMC file: mimo_mmc.tcl
Info: Created 3 library sets
Info: Created 4 constraint modes
Info: Created 4 RC corners
Info: Created 7 analysis views

innovus> report_analysis_views

Analysis View      Mode      Delay Corner
-----
AV_FUNC_SETUP      FUNC      DC_SETUP
AV_FUNC_HOLD        FUNC      DC_HOLD
AV_LP_SETUP         LP        DC_SETUP
AV_LP_HOLD          LP        DC_HOLD
AV_TEST_SETUP       TEST      DC_SETUP
AV_BOOT_SETUP       BOOT      DC_SETUP
AV_FUNC_TYP         FUNC      DC_TYP

innovus> check_timing -verbose
Info: No errors found. 0 unconstrained paths.

innovus> report_timing_summary
# (shows timing summary for all views)

innovus> report_timing -view AV_FUNC_SETUP
# (detailed timing report)
  
```

Key Message:

"A correct MMMC setup, verified with the right commands and free of errors, ensures accurate timing analysis and a **successful signoff**."

✓ Verify Early.
Debug Smartly.
Signoff Confidently!



Complete .MMMC File Example – Part 1 of 3

This part covers: **Library Sets and Constraint Modes**

Same structure
used in real projects!
Clean. Organized.
Production Ready!

1 MMMC File – Part 1 (Library Sets & Constraint Modes)

```

1 #####
2 # MMC File - Part 1 : Library Sets & Constraint Modes
3 #####
4
5 #-----
6 # 1. LIBRARY SETS
7 #-----
8 create_library_set LS_SS \
9   -timing { $TECH_LIB/ss_0p72v_125c.lib }
10 create_library_set LS_TT \
11   -timing { $TECH_LIB/tt_0p80v_25c.lib }
12 create_library_set LS_FF \
13   -timing { $TECH_LIB/ff_0p88v_m40c.lib }
  
```

2 Explanation – Part 1

1 Library Sets

- A library set is a collection of .lib files for a specific PVT corner.
- Here we create three library sets:
 - LS_SS → Slow-Slow (worst case for setup)
 - LS_TT → Typical-Typical (for debug/correlation)
 - LS_FF → Fast-Fast (best case for hold)
- The -timing option specifies the timing .lib file(s).
- You can also include power, noise or other .lib files if required.

Library Set	Typical Condition
LS_SS	Slow corners (e.g., 0.72V, 125°C)
LS_TT	Typical (e.g., 0.80V, 25°C)
LS_FF	Fast corners (e.g., 0.88V, -40°C)

In real projects, each library set may contain multiple .lib files (e.g., delay, memory, IO, PLL, etc.) for the same PVT condition.

```

13
14
15 # 2. CONSTRAINT MODES
16
17 # Functional mode (normal operation)
18 create_constraint_mode FUNC \
19   -sdc_files { $SDC_DIR/func.sdc }
20
21 # Low power mode
22 create_constraint_mode LP \
23   -sdc_files { $SDC_DIR/low_power.sdc }
24
25 # Test mode (scan/ATPG)
26 create_constraint_mode TEST \
27   -sdc_files { $SDC_DIR/test.sdc }
28
29 # Boot mode (initialization)
30 create_constraint_mode BOOT \
31   -sdc_files { $SDC_DIR/boot.sdc }

```

(e.g., logic, memory, IO, PVT) for the same PVT condition.

2 Constraint Modes

- A constraint mode represents one operating mode of the design.
- Each mode can have one or more .sdc files.
- In this example, we create four modes:
 - FUNC** → Normal functional operation
 - LP** → Low power mode (e.g., with power gating)
 - TEST** → Test/Scan mode (for ATPG)
 - BOOT** → Boot/initialization mode
- The -sdc_files option specifies the SDC file(s) for that mode.
- Each mode can have different clocks, constraints, and design intent.

Constraint Mode	Purpose
FUNC	Normal functional operation
LP	Low power mode (retention, power gating)
TEST	Test/Scan mode
BOOT	Startup / initialization

You can specify multiple SDC files, e.g.:
 -sdc_files { \$SDC_DIR/func.sdc \$SDC_DIR/common.sdc }



Key Takeaway:

"Library sets define the **timing environment (PVT)** and constraint modes define the **design intent (SDC)** for each operating mode."



Right Modes.
Right Constraints.
Accurate Timing!



Complete .MMMC File Example – Part 2 of 3

This part covers: **RC Corners and Delay Corners**

Real Setup.
Industry Style.
Well Organized.
Easy to Understand!

1 .MMMC File – Part 2 (RC Corners and Delay Corners)

```

30
31 # MMC File - Part 2 : RC Corners and Delay Corners
32
33 # 3. RC CORNERS
34
35 # Worst RC corner (used for setup)
36 create_rc_corner RC_WORST \
37   -qrc_tech $QRC_TECH_FILE \
38   -T 125 -pre_route_res 1.0 -pre_route_cap 1.0 \
39   -post_route_res 1.0 -post_route_cap 1.0
40
41 # Best RC corner (used for hold)
42 create_rc_corner RC_BEST \
43   -qrc_tech $QRC_TECH_FILE \
44   -T -40 -pre_route_res 0.9 -pre_route_cap 0.9 \
45   -post_route_res 0.9 -post_route_cap 0.9
46
47 # Typical RC corner (used for debug/correlation)
48 create_rc_corner RC_TYP \
49   -qrc_tech $QRC_TECH_FILE \
50   -T 25 -pre_route_res 1.0 -pre_route_cap 1.0 \
51   -post_route_res 1.0 -post_route_cap 1.0
52
53 # 4. DELAY CORNERS
54
55 # Setup delay corner (worst PVT + worst RC)
56 create_delay_corner DC_SETUP \
57   -library_set LS_SS \
58   -rc_corner RC_WORST
59
60 # Hold delay corner (best PVT + best RC)
61 create_delay_corner DC_HOLD \
62   -library_set LS_FF \
63   -rc_corner RC_BEST
64
65 # Typical delay corner (typical PVT + typical RC)
66 create_delay_corner DC_TYP \
67   -library_set LS_TT \
68   -rc_corner RC_TYP

```

2 Explanation – Part 2

1 RC Corners

- An RC corner defines the interconnect parasitics (R and C) using a technology file (e.g., .qrcTech).
- You can specify temperature and scaling factors for pre-route and post-route estimation.
- Typically, a worse RC (higher R/C, high temp) is used for setup, and a best RC (lower R/C, low temp) is used for hold.
- The .qrcTech file contains the process-dependent extraction parameters.

RC Corner	Temp (°C)	Scaling Factor	Used For
RC_WORST	125	1.0 / 1.0	Setup (worst case)
RC_TYP	25	1.0 / 1.0	Debug / Correlation
RC_BEST	-40	0.9 / 0.9	Hold (best case)



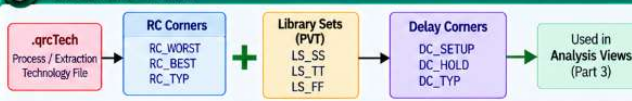
pre_route_res/cap and post_route_res/cap are multipliers used to estimate or scale the R and C values.

2 Delay Corners

- A delay corner combines one Library Set (PVT) + one RC Corner.
- It represents a complete timing environment for analysis.
- In this example:
 - DC_SETUP** → uses worst PVT (LS_SS) + worst RC (RC_WORST)
 - DC_HOLD** → uses best PVT (LS_FF) + best RC (RC_BEST)
 - DC_TYP** → uses typical PVT (LS_TT) + typical RC (RC_TYP)
- These delay corners will be used in the analysis views (Part 3).

Delay Corner	Library Set (PVT)	RC Corner	Used For
DC_SETUP	LS_SS	RC_WORST	Setup analysis
DC_HOLD	LS_FF	RC_BEST	Hold analysis
DC_TYP	LS_TT	RC_TYP	Debug / Correlation

3 Visual Flow – Part 2



Key Takeaway:

"A delay corner = Library Set (PVT) + RC Corner (Parasitics). Choose worst for setup, best for hold, and typical for debug."



Correct Corners.
Accurate Parasitics.
Reliable Timing!



Complete .MMMC File Example – Part 3 of 3

This part covers: **Analysis Views and View Selection (set_analysis_view)**

Final Part!
Complete MMC Flow
Ready for STA
and Signoff!

1 .MMMC File – Part 3 (Analysis Views and View Selection)

```

74
75 # MMC File - Part 3 : Analysis Views and View Selection
76
77 # 5. ANALYSIS VIEWS
78
79 # Functional mode - Setup view
80 create_analysis_view AV_FUNC_SETUP \
81   -constraint_mode FUNC \
82   -delay_corner DC_SETUP
83
84 # Functional mode - Hold view
85 create_analysis_view AV_FUNC_HOLD \
86   -constraint_mode FUNC \
87   -delay_corner DC_HOLD
88
89 # Low power mode - Setup view
90 create_analysis_view AV_LP_SETUP \
91   -constraint_mode LP \
92   -delay_corner DC_SETUP
93
94 # Low power mode - Hold view
95 create_analysis_view AV_LP_HOLD \
96   -constraint_mode LP \
97   -delay_corner DC_HOLD
98
99 # Test mode - Setup view (hold optional)
100 create_analysis_view AV_TEST_SETUP \
101   -constraint_mode TEST \
102   -delay_corner DC_SETUP
103
104 # Boot mode - Setup view (hold optional)
105 create_analysis_view AV_BOOT_SETUP \
106   -constraint_mode BOOT \
107   -delay_corner DC_SETUP
108
109 # Typical view (for debug / correlation, not for signoff)
110 create_analysis_view AV_FUNC_TYP \
111   -constraint_mode FUNC \
112   -delay_corner DC_TYP
113
114 # 6. SELECT ACTIVE VIEWS
115
116 # Setup and hold views used for timing analysis
117 set_analysis_view \
118   -setup { AV_FUNC_SETUP AV_LP_SETUP AV_TEST_SETUP AV_BOOT_SETUP } \
119   -hold { AV_FUNC_HOLD AV_LP_HOLD }

```

2 Explanation – Part 3

1 Analysis Views

- An analysis view is a combination of one constraint mode and one delay corner.
- It represents one complete STA scenario.
- We create separate views for setup, hold, low-power, test, boot, etc.
- You can create additional views as per project requirements.

Each analysis view = Constraint Mode + Delay Corner

2 View Selection

- set_analysis_view specifies which views to use for setup and hold analysis.
- You can specify multiple views (for multi-mode analysis).
- Only the selected views are active in timing analysis.
- Tempus/Innovus will use these active views for all timing reports, optimization and signoff checks.

Typically, you select all functional, low-power, test (for setup) and corresponding hold views.

3 Common Practice

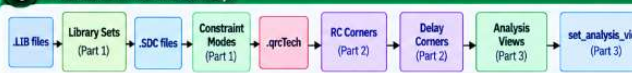
- Use worst corners for setup (e.g., slow PVT + high RC).
- Use best corners for hold (e.g., fast PVT + low RC).
- Include test/boot views if needed (for scan/initialization).
- Keep a typical view for debug and correlation (not mandatory for signoff).

More views can be added (e.g., voltage droop, aging, LVF) depending on project needs.

3 Complete View Summary

Analysis View	Constraint Mode	Delay Corner	Purpose / Usage
AV_FUNC_SETUP	FUNC	DC_SETUP	Functional setup analysis (signoff)
AV_FUNC_HOLD	FUNC	DC_HOLD	Functional hold analysis (signoff)
AV_LP_SETUP	LP	DC_SETUP	Low power setup analysis
AV_LP_HOLD	LP	DC_HOLD	Low power hold analysis
AV_TEST_SETUP	TEST	DC_SETUP	Test/scan setup analysis (hold optional)
AV_BOOT_SETUP	BOOT	DC_SETUP	Boot/initialization setup analysis (hold optional)
AV_FUNC_TYP	FUNC	DC_TYP	Typical view (debug/correlation, not for signoff)

4 Overall MMC Flow (Recap)



Key Takeaway:

"MMMC cleanly separates libraries, constraints, RC corners and analysis views so you can analyze **multiple modes and corners** accurately."



Proper MMC Setup
= Accurate Timing
= Successful Signoff!

